Pocket SDR GNSS SDR Algorithm Description

ver.0.19 2026-07-20

Table of Contents

Scope

This document describes the GNSS SDR receiver implemented by `src/sdr_rcv.c` and the directly connected channel, navigation, and PVT processing functions used by that receiver. It is limited to the real-time tracking receiver path used by `pocket_trk`: RF/IF input handling, signal acquisition, signal tracking, navigation-data decoding, observation generation, and PVT generation.

The document does not describe the standalone post-processing utilities, snapshot positioning, array calibration algorithms except where they are called from the receiver path, or proposed algorithms that are not used by `sdr_rcv.c`.

The intended level of detail is implementation-oriented. Equations are included where they clarify the receiver behavior, but the primary purpose is to connect the algorithmic ideas to the actual Pocket SDR data structures and call paths. For example, the top-level receiver loop is described in terms of `read_data()`, `write_buff()`, `update_srch_ch()`, and `sdr_pvt_udsol()`, and the channel loop is described in terms of `sdr_ch_update()`, `search_sig()`, `track_sig()`, `sdr_nav_decode()`, `sdr_pvt_udnav()`, and `sdr_pvt_udobs()`.

Readers who only need an overview can read Sections 1 and 2 first, then skim the opening paragraphs of Sections 3 to 7. Readers who need to modify the code should read the detailed subsections because many implementation choices, such as the 1 ms receiver cycle, the two-cycle acquisition input length, the single active search channel, and the `SDR_N_CODES` fractional code bank, strongly affect what changes are safe.

The notation used in this document is local to the receiver:

Symbol / name	Meaning in this document
`fs`	IF sample rate in samples per second
`fi`	IF frequency assigned to a receiver channel
`fc`	Carrier replica frequency used during tracking, normally `fi + fd`
`fd`	Estimated Doppler frequency in Hz
`T`	Primary code period for the signal
`N`	Number of samples in one code period, `N = fs * T`
`coff`	Code offset in seconds inside one code period
`adr`	Accumulated Doppler range in carrier cycles
`lock`	Number of tracked code periods after lock
`tow`	Time of week in milliseconds, as decoded or inferred by the channel
`cn0`	Carrier-to-noise-density ratio in dB-Hz
`P`, `E`, `L`, `N`	Prompt, early, late, and noise correlator outputs

1. GNSS Signal Overview

GNSS open-service signals received by Pocket SDR are spread-spectrum signals. Each satellite transmits a carrier at a system-defined RF frequency, modulated by a satellite-specific ranging code. Some signals also carry a navigation data message, while pilot signals carry no data bits but may carry a secondary overlay code.

At receiver baseband, a signal can be modeled as

$$r(t) = A\, d(t)\, c(t-\tau)\, \exp(j(2\pi(f_{\mathrm{IF}} + f_D)t + \phi)) + n(t)$$

where `c(t)` is the primary spreading code, `d(t)` is the navigation data or secondary-code polarity, `tau` is the code delay, `f_D` is Doppler, and `n(t)` is noise plus interference. The SDR receiver estimates `f_D`, `tau`, carrier phase, code phase, and eventually navigation time, then converts them into observables.

Pocket SDR identifies signals by string IDs such as `L1CA`, `L2CM`, `L5I`, `E1B`, `E5AQ`, `B1I`, and `I5S`. Code generation, nominal carrier frequency, code period, secondary-code generation, BOC handling, and RINEX code mapping are selected from this signal ID.

The receiver supports the signal families listed in the command reference:

System	Examples
GPS / QZSS	`L1CA`, `L1CB`, `L1CD`, `L1CP`, `L2CM`, `L5I`, `L5Q`
QZSS augmentation	`L1S`, `L5SI`, `L5SQ`, `L5SIV`, `L5SQV`, `L6D`, `L6E`
GLONASS	`G1CA`, `G2CA`, `G1OCD`, `G1OCP`, `G2OCP`, `G3OCD`, `G3OCP`
Galileo	`E1B`, `E1C`, `E5AI`, `E5AQ`, `E5BI`, `E5BQ`, `E5ABQ`, `E6B`, `E6C`
BeiDou	`B1I`, `B1CD`, `B1CP`, `B2I`, `B2AD`, `B2AP`, `B2BI`, `B3I`
NavIC	`I1SD`, `I1SP`, `I5S`, `ISS`
SBAS	`L1CA`, `L5I`, `L5Q`

The implementation uses one receiver channel for one `(signal, PRN, RF channel)` combination. Each channel owns its generated primary code, secondary code, acquisition state, tracking state, navigation decoder state, and current observables.

1.1 Signal Components Used by the Receiver

The receiver separates each GNSS signal into components that are handled by different modules:

- carrier frequency and Doppler are handled by channel acquisition and tracking;
- primary spreading codes are generated by `sdr_gen_code()`;
- secondary or overlay codes are generated by `sdr_sec_code()`;
- code periods and code lengths are obtained from `sdr_code_cyc()` and `sdr_code_len()`;

- navigation-message framing and FEC are handled by `sdr_nav_decode()`;
- signal-to-observation mapping is handled by `sig2code()` in the PVT module.

The software does not keep a general symbolic signal model at run time. Instead, it resolves the model into concrete code arrays, code periods, IF frequencies, and decoder functions when a channel is created. This is why a receiver channel contains both generic fields (`fd`, `coff`, `adr`, `cn0`) and signal-specific resources (`code`, `sec_code`, `len_code`, `len_sec_code`, decoder selection by `sig`).

The primary code controls acquisition and the base tracking epoch. The secondary code, if present, is handled after lock because its phase is usually unknown until enough prompt correlations are collected. This separation is important for long pilot codes and modernized signals. It allows the receiver to lock to a primary-code epoch first, then establish secondary-code timing and navigation time later.

1.2 Data-Bearing and Pilot Signals

Pocket SDR uses the same basic acquisition and tracking machinery for data-bearing and pilot signals, but navigation-time recovery differs.

For data-bearing signals, prompt in-phase correlations are converted into soft binary symbols. Each symbol keeps both the sign decision and a limited reliability value. The decoder looks for symbol boundaries, preambles, known subframe patterns, and CRC/parity success. FEC decoders can consume the soft values directly, while parity-only and CRC-check paths make local hard decisions when they need bits. Once a valid navigation frame is decoded, the channel can obtain week number, TOW, frame type, and raw message payload. These fields are used by `sdr_pvt_udnav()` to update ephemerides and by `sdr_pvt_udobs()` to generate absolute pseudorange.

For pilot signals, there may be no navigation frame to decode. The channel can still track carrier and code phase, but it may only know time modulo a code or secondary-code interval. In the implementation, such cases are represented by `tow_v = 2`, meaning that the channel has timing information with unresolved ambiguity. PVT observation generation first forms a folded pseudorange for such channels. Only selected short-period ambiguous signals are later resolved against a companion signal or invalidated if no companion range is available.

The receiver also supports paired data/pilot designs. Examples include GPS L5 I/Q, Galileo E5a I/Q, BeiDou B1C D/P, and Galileo E5 AltBOC `E5ABQ`. In these cases, one component may be used for robust tracking while another carries the navigation frame. The actual pairing is not represented by a separate high-level object. Instead, channels are independent, and the PVT layer merges observations and navigation data through satellite ID, signal code, and epoch time.

1.3 Time Scales and Epochs

The receiver uses several time notions at once:

- input sample time, expressed as `ix * SDR_CYC` in the receiver thread;
- channel tracking time, stored in `ch->time`;

- code epoch count, stored in `ch->lock`;
- decoded GNSS week/TOW, stored in `ch->week`, `ch->tow`, and `ch->tow_v`;
- observation epoch time, stored in `pvt->time` and `pvt->ix`;
- solution time, stored in the RTKLIB `sol_t` object.

The 1 ms receiver cycle is a software scheduling unit, not necessarily a signal code period. Most open-service GNSS ranging codes used by the receiver are 1 ms or an integer multiple of 1 ms, so the scheduling model is convenient. Some signals have longer code periods; their channel thread advances by multiple receiver cycles. Very short sub-ms signals do not fit the current channel model well unless they are handled inside a larger acquisition/tracking design.

The channel call

```
sdr_ch_update(ch, th->ix * SDR_CYC, buffer, rcv->N * (th->ix % MAX_BUFF))
```

passes the nominal time at the start of the channel update and the circular buffer index for the current code period. The channel then advances its carrier and code states by the elapsed time from the previous channel update.

1.4 IF Representation

After input conversion, the receiver stores IF samples as `sdr_cpx8_t`. The sample values are compact signed integer I/Q pairs, not floating-point complex samples. Carrier wipeoff converts the compact buffer into `sdr_cpx16_t` working samples. In the standard correlator this conversion is fused with the correlation itself: each cache-sized chunk is carrier-wiped and immediately correlated, so no full-length baseband buffer is written. The L6 CSK path (`L6D` / `L6E`) and the complex-code path (`E5ABQ`) still convert a full code period into a `sdr_cpx16_t` working buffer. Correlator outputs are accumulated in `sdr_cpx_t`, which is the floating-point complex type used by the FFT and loop discriminators.

This representation is central to the implementation. It reduces memory bandwidth in the circular buffers, keeps lookup-table unpacking cheap, and allows the standard correlator to use integer-vector operations. It also means that automatic scale control is implemented before the internal IF buffer: the receiver periodically estimates the input statistics and regenerates LUTs so that the compact sample values occupy a useful dynamic range.

1.5 Signal Selection by RF Frequency

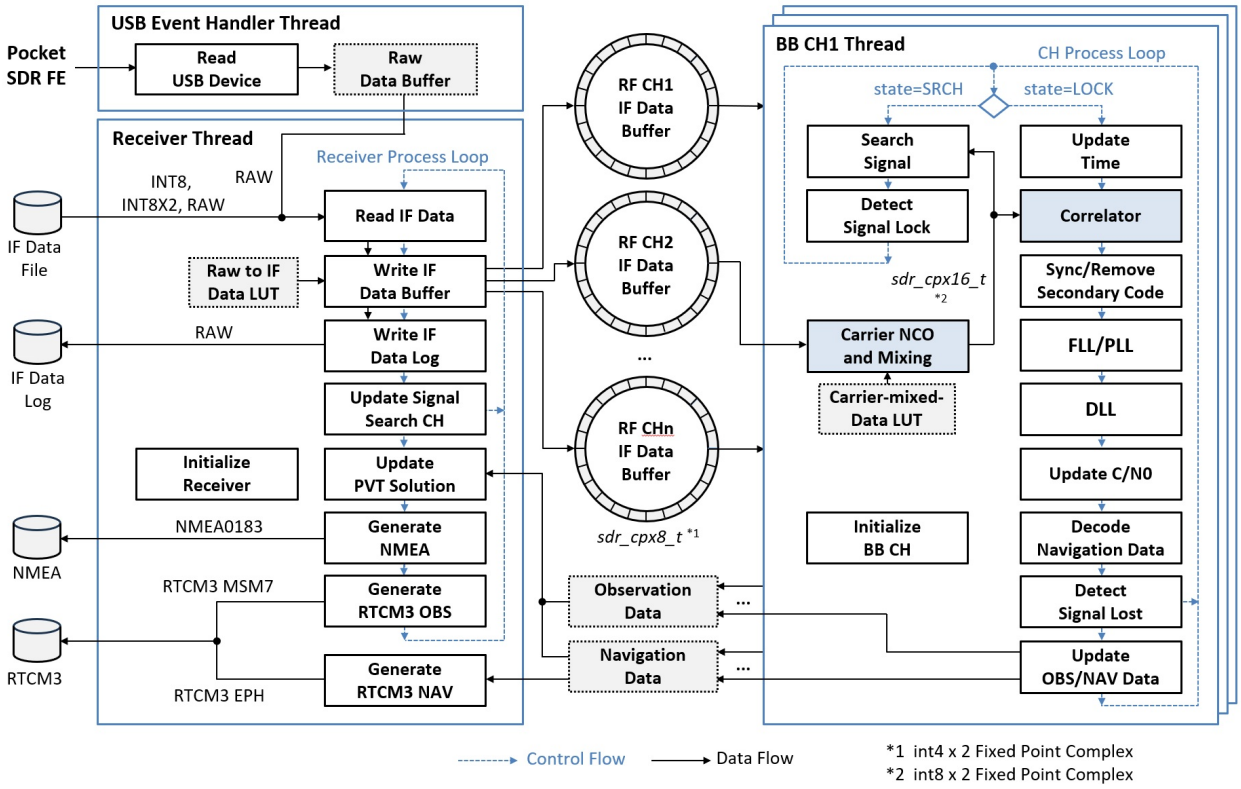
When the user does not explicitly bind a signal to an RF channel, `set_rfch()` selects a channel from the nominal signal frequency and the configured front-end LO frequencies. For packed Pocket SDR FE formats, this maps high-frequency and low-frequency signals to appropriate RF paths. For generic single-channel IQ input, there is only one RF channel and the assigned IF frequency is either `freq - fo` or zero-IF.

For GLONASS FDMA signals, `sdr_shift_freq()` applies the frequency-channel offset. That is why the receiver uses the PRN argument as FCN for `G1CA` and `G2CA` acquisition and tracking. The same shifted-frequency helper is used when the channel stores both RF carrier frequency `fc` and IF frequency `fi`.

2. Overall GNSS SDR Architecture

2.1 Pocket SDR Architecture

The figure below gives an overall view of the receiver: the USB event-handler and receiver threads, the per-RF-channel circular IF buffers, and the per-baseband-channel processing loop.



Pocket SDR Receiver Architecture and Processing Flow

2.2 Receiver Objects

The top-level object is `sdr_rcv_t`. It contains:

- input source state: device type, file/stream/device pointer, IF format, sampling frequency, LO frequencies, and RF channel definitions;
- circular IF buffers, one per physical RF channel plus optional array-derived channels;
- one `sdr_ch_th_t` channel thread per baseband receiver channel;
- one `sdr_pvt_t` object for navigation data, observations, and PVT solution;
- output streams for NMEA, RTCM3, log, and raw IF data.

The receiver operates on a fixed processing cycle `SDR_CYC = 1 ms`. The main receiver thread reads one 1 ms block from the input, converts it into internal complex 8-bit IF samples, writes it to circular buffers, starts or advances signal searches, and updates PVT epochs. Channel threads consume the same circular buffers at each signal's code period.

2.3 RF Front-End and SDR Partition

The RF front-end is responsible for analog GNSS reception, amplification, filtering, frequency conversion, sampling, and transport to the host. Pocket SDR FE devices expose packed raw formats for 2, 4, or 8 RF channels. Alternative front-ends can be used through file input, generic streams, or SoapySDR.

The SDR software starts at digital IF samples. Its front-end-related work is:

- read samples from file, USB FE, TCP/RTKLIB stream, or SoapySDR;
- unpack packed FE formats (`RAW8`, `RAW16`, `RAW16I`, `RAW32`) or normalize generic `INT8`, `INT8X2`, `CS8`, `CS16` formats;
- convert samples to internal `sdr_cpx8_t` complex samples through lookup tables;
- for real-sampling RF channels, apply an `fs/4` real-to-complex conversion and optional low-pass filtering;
- select an RF channel automatically from signal RF frequency and LO frequency, or explicitly through `-RFCH`;
- optionally combine multiple RF channels into array/beam channels.

Signal processing after this point is pure SDR: acquisition, tracking, navigation decoding, observation generation, and PVT.

2.4 Runtime Data Flow

The receiver has two levels of threads.

1. The receiver thread reads IF data and writes buffers:

```
read_data() -> write_buff() -> update_srch_ch() -> sdr_pvt_udsol()
```

2. Each channel thread waits until enough buffered data is available and then advances the channel:

```
sdr_ch_update()
    if SRCH: search_sig()
    if LOCK: track_sig()
if nav updated: sdr_pvt_udnav()
sdr_pvt_udobs()
```

The channel state machine has three states:

- `SDR_STATE_IDLE`: waiting for the scheduler to start acquisition.

- `SDR_STATE_SRCH`: running acquisition.
- `SDR_STATE_LOCK`: tracking, decoding navigation data, and producing observables.

The receiver scheduler allows at most one channel to be in the active search state at a time. This limits acquisition CPU load while all locked channels continue tracking in parallel.

2.5 Receiver Initialization

The main receiver object is created by `sdr_rcv_new()`. The function converts user-level receiver configuration into the concrete objects used by the runtime loop. Its main steps are:

1. Parse receiver options that affect global channel behavior, such as array channel count, E5 AltBOC group-delay offset, and BOC bump-jump threshold.
2. Store IF format, sample rate, RF channel count, LO frequencies, sampling modes, and bit widths.
3. For each requested `(signal, PRN)` pair, select one or more RF channels and compute the channel IF frequency.
4. Create a channel thread object with `ch_th_new()`, which internally creates an `sdr_ch_t` channel by calling `sdr_ch_new()`.
5. Allocate circular IF buffers for all RF channels and optional array-derived channels.
6. Create the PVT object and initialize mutexes and status fields.

Channel creation is intentionally front-loaded. Each `sdr_ch_t` allocates its primary code, secondary code, acquisition FFT, tracking code bank, navigation decoder state, and work buffers before the receiver starts. This keeps the real-time tracking loop free from large allocations except for acquisition accumulation buffers, which are allocated only while a channel is in search.

The initialization path also handles real-sampling channels. If an RF channel is configured as I-only sampling, the channel IF frequency is shifted by `-fs/4` because `write_buff()` applies a corresponding `fs/4` digital mixing pattern during sample unpacking. After this adjustment, later channel processing can use the same complex-carrier wipeoff path for both I-only and IQ inputs.

2.6 Receiver Open Paths

The public open functions differ mainly in how they obtain IF metadata and a data pointer:

Function	Input source	Metadata source
`sdr_rcv_open_dev()`	Pocket SDR USB FE	Device query plus optional configuration file
`sdr_rcv_open_sdev()`	SoapySDR device	Soapy driver settings and requested format
`sdr_rcv_open_file()`	Local IF file	Command-line parameters or `.tag` file
internal stream open path	RTKLIB stream	Caller-provided format and sample settings

After metadata is known, all open paths converge on `sdr_rcv_new()` and `sdr_rcv_start()`. This is important for algorithm consistency: acquisition, tracking, navigation decoding, observation generation, and PVT do not depend on whether the samples came from hardware, a file, or a stream.

The `.tag` file mechanism is part of that convergence. For file replay, a tag file can override format, sample rate, LO frequencies, IQ mode, and bit width. For raw IF logging from a live device, `write_raw_tag_file()` writes equivalent metadata so the recorded samples can be replayed with the same interpretation.

2.7 Input Formats and Internal Buffers

The receiver accepts both generic IF files and packed FE output. The internal goal is always the same: produce one complex sample stream per RF channel.

Format	Meaning	Internal conversion
`INT8`	signed 8-bit real samples	I-only samples, Q set to zero
`INT8X2`	interleaved signed 8-bit IQ with inverted Q convention	LUT-scaled I and negated Q
`CS8`	interleaved signed 8-bit IQ	LUT-scaled I and Q
`CS16`	interleaved signed 16-bit IQ	separate I/Q LUTs to compact 8-bit values
`RAW8`	packed Pocket SDR FE 2-channel raw	unpack per RF channel
`RAW16`	packed Pocket SDR FE 4-channel raw	unpack per RF channel
`RAW16I`	packed Spider SDR FE 8-channel raw	unpack I-only channels
`RAW32`	packed Pocket SDR FE 8-channel raw	unpack per RF channel

For packed FE formats, a byte contains quantized samples for multiple channels. `write_buff()` uses per-channel LUTs to unpack the selected bits and write them to the correct `sdr_buff_t`. For real-sampled channels, a four-phase LUT implements the `fs/4` quadrature conversion:

```
phase 0:  I,  0
phase 1:  0, -I
phase 2: -I,  0
phase 3:  0,  I
```

This conversion is cheap because the sample byte and the phase form a single lookup index. The phase is derived from the absolute receiver cycle and sample offset, so it remains continuous across circular-buffer wraparound.

2.8 Receiver Thread Loop

The receiver thread is the only thread that reads the input source. Its loop can be summarized as:

```
for ix = 0, 1, 2, ... while receiver is running:
    read 1 ms of raw data
    write internal IF buffers
    optionally write raw IF stream
    update acquisition scheduler
    update PVT epoch
    update input statistics and sample scaling
    sleep if replaying a file faster than the requested time scale
```

The loop also periodically updates buffer usage, input data rate, and time logs. For live devices, `sdr_dev_start()` or `sdr_sdev_start()` is called before the loop and the corresponding stop function is called after the loop exits.

The receiver loop is deliberately simple. It does not run acquisition or tracking itself. It only produces consistent IF buffers and schedules channel state changes. This split keeps input timing independent from the potentially bursty acquisition workload.

2.9 Channel Thread Loop

Each channel thread owns one receiver channel. It does not read the input source; it only reads from the RF channel buffer selected by `ch->rf_ch`. A channel thread computes

$$n = \frac{N_{\mathrm{ch}}}{N_{\mathrm{rcv}}}$$

where (N_{\mathrm{rcv}}) corresponds to `rcv->N`, the number of samples per 1 ms receiver cycle, and (N_{\mathrm{ch}}) corresponds to `ch->N`, the number of samples per code period. The thread advances by `n` receiver cycles at a time. It waits until at least two code periods of data are present:

```
while th->ix + 2 * n <= rcv_ix:
    update channel at th->ix
    th->ix += n
```

The two-period availability requirement is especially important for acquisition. `search_sig()` passes `2 * ch->N` samples into `sdr_search_code()` so that zero-padded correlation can search a complete code-offset interval without losing samples near the boundary.

After each channel update, the thread transfers navigation and observation state to the PVT object. Navigation updates happen only when a decoder sets `ch->nav->stat`. Observation updates are checked every channel epoch, but an observation is generated only at the current PVT epoch and only if the channel has valid timing.

2.10 Shared-State Boundaries

The implementation uses mutexes only at the boundaries where shared state must be consistent:

- the receiver buffer index `rcv->ix` is protected by `rcv->mtx`;
- channel tracking state is protected by `ch->mtx` during `track_sig()`;
- PVT navigation, observation, and solution state is protected by `pvt->mtx`;
- array channel beam state is updated under receiver locking when IF buffers are combined.

The IF circular buffers themselves are written by the receiver thread and read by channel threads. Synchronization is achieved by the monotonic buffer index: the receiver writes a complete 1 ms block before publishing the new index, and channel threads only process data whose cycle index is safely behind that published index.

2.11 Processing Latency

Latency comes from several sources:

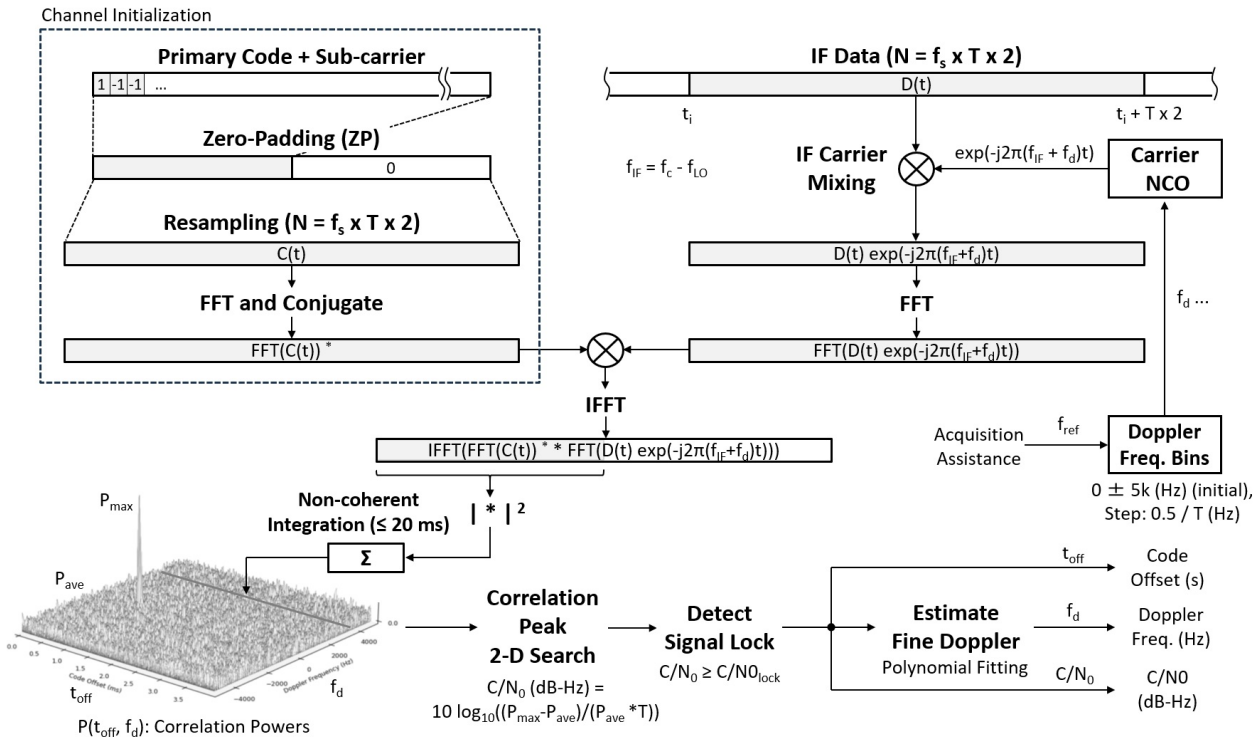
- the circular-buffer delay between the receiver writer and channel readers;
- acquisition non-coherent integration time, normally 20 ms;
- navigation pull-in time before symbol/frame decoding, normally 1.5 s;
- observation epoch waiting and lag allowance;
- PVT solution computation and output streams.

The PVT layer records `pvt->latency` as the difference between the current input cycle and the epoch cycle. If not all channels have reached the epoch, the PVT update waits until either all channels have reported or `sdr_lag_epoch` is exceeded. This avoids blocking solution generation indefinitely because of one slow or lost channel.

3. Signal Acquisition Algorithm

3.1 Data Flow of Signal Acquisition

The figure below shows the acquisition data flow: parallel code-phase / Doppler FFT search with non-coherent integration, peak detection, and C/N0 estimation.



f_s: Sampling Rate (sps), T: Primary Code Cycle (s), f_{IF}: IF Carrier Freq. (Hz), f_d: Doppler Freq. (Hz), t_{off}: Code Offset (s), $C/N0_{lock}$: Lock Threshold (dB-Hz)
f_c: Carrier Freq. (Hz), f_{LO}: Local Oscillator (PLL) Freq. (Hz)

Data Flow of Signal Acquisition

3.2 Search Scheduling

The receiver calls `update_srch_ch()` once per 1 ms input block. If the IF buffer is close to full, no new search is started. If another channel is already searching, the scheduler waits. Otherwise, it rotates through idle channels and starts a search if one of these conditions is true:

- re-acquisition is possible after recent lock loss;
- Doppler assistance is possible from another locked signal of the same satellite;
- the signal code period is short enough for direct acquisition (`T <= sdr_max_acq`, default `4 ms`) and the channel is enabled for search.

Re-acquisition uses the channel's last tracked Doppler for up to 60 s after loss, provided the previous lock lasted at least 2 s. Assisted acquisition uses a locked signal from the same satellite and scales the tracked Doppler by the carrier-frequency ratio.

3.3 Parallel Code Phase Search

Acquisition uses a parallel-code FFT search. For each Doppler bin:

1. Mix the IF block by `fi + fd`.
2. Compute FFT-based circular correlation against the precomputed code FFT.
3. Add the correlation power to a non-coherent accumulation buffer.

The acquisition code FFT is generated at channel creation. It is zero-padded to `2 * N` samples for normal acquisition, where `N = fs * T` and `T` is the code period. Doppler bins are generated from the code period and configured maximum Doppler. If external Doppler assistance is available, the normal bin list is usually replaced by three bins centered on the assisted Doppler:

$$f_d \in \{f_{d,\mathrm{ext}} - \frac{0.5}{T}, f_{d,\mathrm{ext}}, f_{d,\mathrm{ext}} + \frac{0.5}{T}\}$$

Fast-search aiding can request a wider assisted window by setting `fd_ext_n`; the same bin spacing is then used across the requested number of bins around `fd_ext`.

The receiver distinguishes normal and Doppler-assisted searches with `fd_ext_valid`, so an exact zero-Hz Doppler aid is valid. Normal acquisition accumulates until `n_sum * T >= sdr_t_acq` (default `20 ms`). Assisted acquisition uses `sdr_t_acq_ext` (default `100 ms`) because its much smaller Doppler search space makes longer integration affordable. It then searches the Doppler/code grid for the maximum power and estimates C/N0 from the peak-to-average ratio:

$$C/N_0 = 10\log_{10}(\frac{P_{\max} - P_{\mathrm{ave}}}{P_{\mathrm{ave}}\,T})$$

For normal acquisition, tracking starts if the result exceeds `sdr_thres_cn0_l` (default `34 dB-Hz`). An assisted search instead uses

$$\max(C/N_{0,\mathrm{ext}}, C/N_{0,\mathrm{lost}} + 1\ \mathrm{dB})$$

where `sdr_thres_cn0_ext` defaults to `30 dB-Hz`, and the lost threshold is `sdr_thres_cn0_u` or the L6-specific threshold. The margin prevents immediate loss caused by the C/N0 gate; PLI and loop-dynamic limits remain separate. Otherwise the channel returns to idle.

3.4 Special Acquisition Cases

`E5ABQ` acquisition uses an E5aQ-only AltBOC complex replica so that the signal can be found before secondary-code alignment is known. The full E5aQ/E5bQ pilot combination is used later in tracking after secondary-code sync.

QZSS `L6D` and `L6E` acquire like ordinary signals, but their CSK symbol structure also requires a code-shift search after lock; tracking runs a separate small chip-domain FFT correlation for that purpose (see 4.16).

3.5 Acquisition State Data

The acquisition state is stored in `sdr_acq_t`:

Field	Role
`code_fft`	FFT of the local code replica used by PCPS
`fds`	Normal Doppler search bin list
`len_fds`	Number of Doppler bins
`fd_ext`	Optional external Doppler assistance
`fd_ext_valid`	Validity flag for `fd_ext`, including an exact 0 Hz aid
`fd_ext_n`	Assisted Doppler-bin count (0 selects the default three bins)
`P_sum`	Non-coherent sum of correlation powers
`n_sum`	Number of coherent code-period searches accumulated

`P_sum` is allocated lazily when the channel enters search. It is released when the acquisition decision is made. This keeps idle channels light even if many channels are configured. The permanent acquisition memory is mostly the code FFT and Doppler-bin list.

The code FFT is generated by `sdr_gen_code_fft()` for ordinary signals. The function resamples the primary code to the channel sample rate, applies optional zero padding, performs the FFT, and stores the complex conjugate of the result. The later acquisition correlator can therefore compute

$$R = \mathrm{IFFT}(\mathrm{FFT}(x)\,\mathrm{conj}(\mathrm{FFT}(c)))$$

by multiplying the data FFT by `code_fft`.

3.6 Doppler Bin Spacing

The bin spacing is tied to the coherent integration length. For a coherent integration interval `T`, a Doppler error of roughly `1/T` Hz produces one cycle of phase rotation across the integration. The implementation therefore creates bins through `sdr_dop_bins(T, 0, sdr_max_dop, &len_fds)`, and assisted acquisition uses half-bin spacing around the assisted Doppler.

This makes acquisition behavior signal-dependent. A 1 ms signal uses wider Doppler bins than a 4 ms coherent search because phase rotation accumulates for a shorter time. Long code periods produce finer Doppler spacing and more bins for the same maximum Doppler range. That is one reason the receiver limits direct acquisition by `sdr_max_acq`: very long code periods are expensive both in code phase and Doppler dimensions.

The configured maximum Doppler is normally symmetric around zero. For live tracking, this value must cover satellite line-of-sight velocity, receiver oscillator error, and any IF frequency error. When a same-satellite channel is already locked, the receiver can avoid the wide search because oscillator error and satellite Doppler are common between frequencies to first order.

3.7 Coherent and Non-Coherent Integration

Each call to `sdr_search_code()` performs one coherent code-period correlation for every Doppler bin. The coherent result is converted to power and added to `P_sum`. After enough calls, the non-coherent integration interval reaches `sdr_t_acq` for a normal search or `sdr_t_acq_ext` for an assisted search.

The distinction is:

$$\begin{aligned} &\text{coherent} : \text{complex correlation over one code period} \\ &\text{non-coherent} : \sum_i |R_i|^2 \text{ over multiple code periods} \end{aligned}$$

Non-coherent accumulation is robust to data-bit sign changes and secondary-code polarity changes because it discards phase. The cost is lower sensitivity than a fully coherent multi-ms integration when the data/secondary polarity is known. The implementation chooses robustness and simplicity for acquisition because tracking and navigation decoding will resolve polarity after lock.

For signals with a primary code period shorter than the desired acquisition time, many code periods are accumulated. For a 1 ms code and a default 20 ms acquisition interval, the receiver accumulates 20 coherent searches. For a 4 ms code, it accumulates five searches.

3.8 Acquisition Metric

After accumulation, `sdr_corr_max()` computes an average power over the searched Doppler/code grid and finds the maximum peak. It converts the excess peak power over the average into a C/N0-like metric. This is not a full statistical constant-false-alarm detector. It is a practical acquisition metric that works with the receiver's tracking threshold and status display.

The acquisition decision has two outputs:

- if `cn0` reaches the effective normal or assisted threshold, the channel starts tracking;
- otherwise the channel sleeps briefly, returns to idle, and may be searched again later by the scheduler.

The brief sleep after a failed search intentionally releases CPU time. A large configuration can include many idle channels, and without this release a stream of failed acquisition attempts could reduce tracking margin.

3.9 Code-Offset Interpretation

The acquisition code offset is returned as a sample index inside the searched correlation vector. The channel converts it to seconds:

$$\mathrm{coff} = \frac{i_{\mathrm{code}}}{f_s}$$

At the start of tracking, `coff` is the estimated code offset modulo one primary code period. The tracking loop then refines it continuously through the DLL and carrier-aided code update. Because acquisition uses `2 * N` samples but searches only the first `N` code offsets for peak selection, the initial offset is kept inside one code period.

The Doppler estimate is refined by `sdr_fine_dop()`, which fits the local power shape around the detected Doppler bin. This gives a better initial frequency than the discrete bin center and reduces the FLL pull-in burden.

3.10 Acquisition Pseudocode

The implemented acquisition loop for one channel can be expressed as:

```
assisted = fd_ext_valid
if assisted:
    if fd_ext_n is set:
        fds = fd_ext-centered bins with fd_ext_n entries at 0.5/T spacing
    else:
        fds = [fd_ext - 0.5/T, fd_ext, fd_ext + 0.5/T]
else:
    fds = normal Doppler bins

if P_sum is not allocated:
    allocate P_sum for len(fds) x 2N

sdr_search_code(code_fft, T, buffer, ix, 2N, fs, fi, fds, P_sum)
n_sum += 1

T_acq = sdr_t_acq_ext if assisted else sdr_t_acq
threshold = max(sdr_thres_cn0_ext, lost_threshold + 1) if assisted
            else sdr_thres_cn0_l

if n_sum * T >= T_acq:
    cn0, fd_index, code_index = peak_search(P_sum)
    if cn0 >= threshold:
        fd = fine_doppler(P_sum, fd_index)
        coff = code_index / fs
        start_track(fd, coff, cn0)
    else:
        return to idle
    free P_sum
```

This pseudocode is useful when considering changes. Any alternative acquisition method still has to produce the same channel-start outputs: Doppler `fd`, code offset `coff`, and initial C/N0. It also has to cooperate with the receiver's single-search scheduler and buffer-lag limits.

3.11 Failure Modes and Re-Acquisition

Acquisition can fail for expected reasons:

- the satellite is below the antenna horizon;
- the chosen RF channel does not contain the signal band;
- the IF frequency or sampling metadata is wrong;
- Doppler is outside the configured range;
- the integration time is too short for the received C/N0;
- a strong interferer raises the average correlation power;
- a long code or data transition reduces coherent gain.

The receiver does not blacklist failed channels. It cycles through idle channels, so a failed channel can be searched again later. If a channel had a valid previous lock, re-acquisition starts with the old Doppler for a limited time. If another frequency from the same satellite is locked, assisted acquisition uses that channel's Doppler. These two mechanisms make loss recovery much faster than cold acquisition.

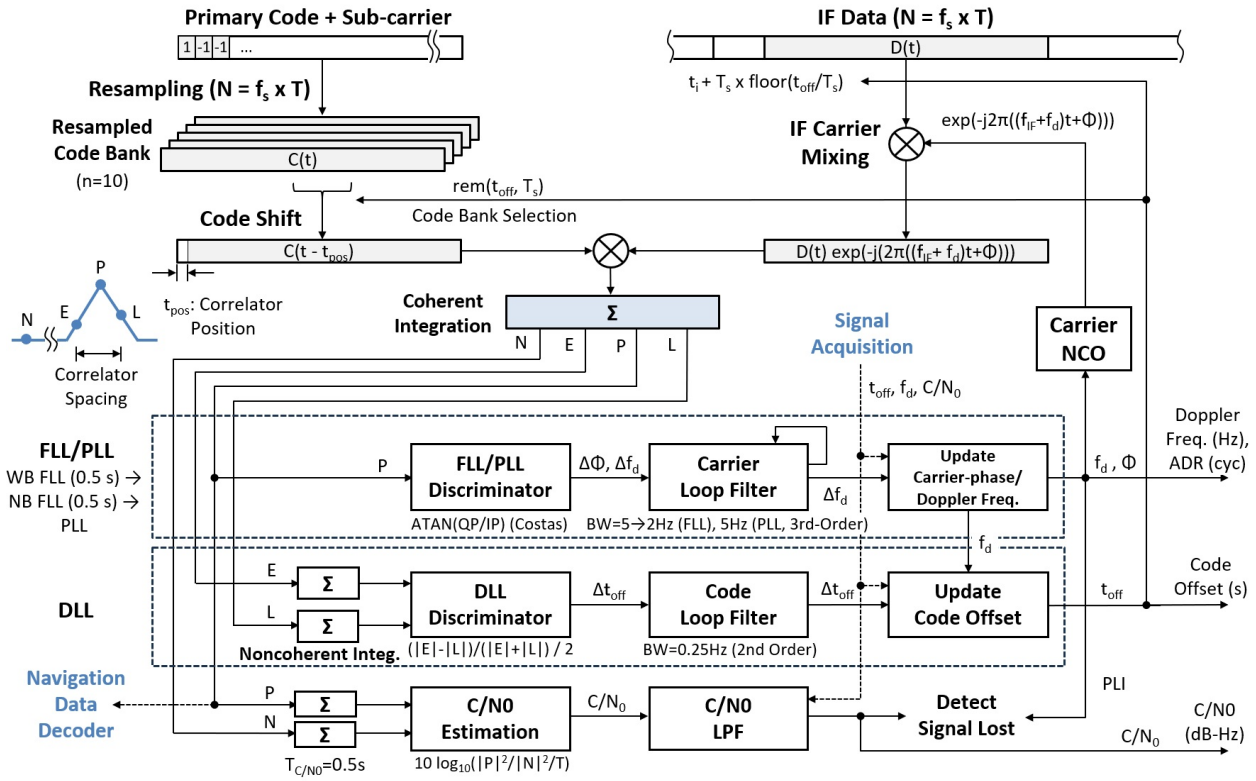
3.12 Relationship to `pocket_acq`

The receiver uses the same core PCPS helper as offline acquisition, but the real-time receiver path differs in scheduling and state management. `pocket_acq` can spend all CPU on one file segment and one set of PRNs. `sdr_rcv.c` must protect ongoing tracking, keep input buffers from overflowing, and share CPU with navigation and PVT generation. The single active search channel and the buffer-usage guard are therefore part of the algorithm, not just implementation details.

4. Signal Tracking Algorithm

4.1 Data Flow of Signal Tracking

The figure below shows the tracking data flow: carrier and code wipe-off, the early/prompt/late/noise correlators, the FLL/PLL and DLL loops, C/N0 estimation, and loss detection.



E/P/L/N: Early/Prompt/Late/Noise Correlation, f_s: Sampling Rate (sps), T_s: Sampling Cycle (s), T: Primary Code Cycle (s), f_{IF} : IF Carrier Freq. (Hz), f_d: Doppler Freq. (Hz), t_{off} : Code Offset (s), Φ: Carrier-phase (cyc), $Δf_d$: Doppler Error (Hz), $Δt_{off}$: Code Offset Error (s), ΔΦ: Carrier-phase Error (cyc)

Data Flow of Signal Tracking

4.2 Tracking Replicas and Correlators

At channel creation, the receiver builds a tracking replica bank. For ordinary real spreading codes, the code is resampled at `fs` for `SDR_N_CODES = 10` fractional code phases and stored as 1-byte `int8_t` samples (values -1/0/1), which halves the bank footprint compared to a complex representation. Each tracking epoch selects the bank corresponding to the current fractional code offset.

The default correlator positions are:

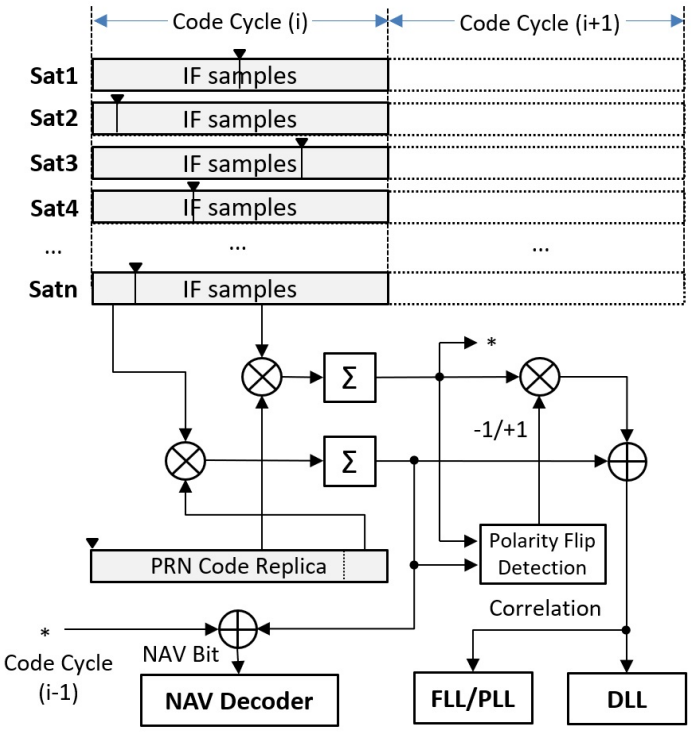
- Prompt `P`;

- Early `E` and late `L`, spaced by `sdr_sp_corr` chips, default `0.25`;
- noise correlator `N`, located far away at a fixed sample offset;
- optional very-early and very-late correlators for BOC false-lock detection.

For most signals, tracking uses the time-domain standard correlator with fused carrier wipeoff: the IF samples are carrier-wiped and correlated with prompt, early, late, noise, and optional BOC monitor replicas chunk by chunk in a single pass, which keeps the working samples in cache and avoids re-reading a full-length baseband buffer for each correlator. The correlator splits the code period at the code wrap point and detects a polarity flip between the two halves, which reduces the impact of data-bit transitions inside the integration interval. The correlator logic, including the cross-epoch combination of the two halves, is shown in the figure below (detailed in §4.10–§4.11).

For `E5ABQ`, the correlator uses complex spreading-code replicas and selects between three precomputed banks: E5aQ only before secondary-code sync, and two E5aQ/E5bQ pilot combinations after sync depending on the relative secondary code polarity.

For `L6D` and `L6E`, the CSK symbol is detected by a chip-domain FFT correlation over the candidate code shifts, and the correlator outputs are then produced by the standard correlator against the CSK-shifted code (see 4.16).



Correlator Logic

4.3 Carrier Tracking

Tracking maintains Doppler `fd`, accumulated Doppler range `adr`, and carrier phase. The carrier replica frequency is

$$f_c = f_i + f_d$$

At each code period:

- accumulated Doppler is advanced by `fd * tau`;
- code offset is carrier-aided by `-fd / carrier_frequency * tau`;
- IF samples are mixed by the current carrier replica;
- prompt correlation is stored in a prompt history buffer.

Carrier tracking uses a pull-in FLL followed by a PLL.

During the first `T_FPULLIN = 1.0 s`, an FLL discriminator compares the current and previous prompt correlations using dot and cross products. It uses a wide bandwidth first, then a narrow bandwidth. After pull-in, a third-order PLL is used. The normal path uses the channel's Costas/non-Costas discriminator every code period. A supported pilot switches to secondary-code-wiped coherent prompt sums and a four-quadrant `atan2` discriminator after secondary-code sync, as detailed in 4.13.

4.4 Code Tracking

Code tracking uses a non-coherent early-minus-late DLL. Correlation powers are summed over `sdr_t_dll` (default `20 ms`). The code discriminator is

$$e_{\text{code}} = \frac{\sqrt{E} - \sqrt{L}}{\sqrt{E} + \sqrt{L}} \frac{0.5\,T}{N_{\text{code}}}$$

A second-order loop filter updates the code offset. The prompt I sign is used for data wipeoff in the accumulated in-phase monitor values.

If the code offset crosses the code-period boundary, the receiver wraps it back to `[0, T)` and adjusts lock count and TOW consistently.

4.5 Secondary-Code Sync and C/N0

For signals with secondary codes, the channel waits until navigation pull-in time (`T_NPULLIN = 1.5 s`), then correlates the prompt history with the secondary-code sequence. The sync test uses the ratio between the signed secondary-code correlation and the average prompt magnitude, so weak or occasional sign errors do not force a hard reject. Once synchronized, the secondary-code polarity is removed from all correlator outputs. If the average prompt magnitude falls below a loss threshold at a secondary-code boundary, sync is cleared.

C/N0 is estimated every `T_CN0 = 0.5 s` from prompt power and the displaced noise correlator. The noise power contained in the prompt is subtracted before conversion to dB-Hz:

$$P_S = \max(P_{\text{prompt}} - P_{\text{noise}}, 0.01P_{\text{noise}})$$
$$C/N_0 = 10\log_{10}(\frac{P_S}{P_{\text{noise}}\,T})$$

The 1% floor avoids a logarithm-domain error when the estimated signal power is zero or negative. The estimate is low-pass filtered.

At the same `T_CN0` boundary the receiver also updates a carrier lock indicator (PLI), a squaring (Van Dierendonck) detector that is independent of received power and so catches the case where the carrier loses phase lock while prompt power stays high:

$$\text{PLI} = \frac{\text{sumD}}{\text{sumPs}} = \frac{\sum(I_P^2 - Q_P^2)}{\sum(I_P^2 + Q_P^2)} \approx \cos 2\bar{\phi}$$

PLI tends to `+1` under phase lock and to `0` when the carrier spins or there is only noise. The normal path accumulates per-period prompts. A synchronized pilot accumulates the coherent prompt sums used by its PLL, so numerator and denominator always use the same integration interval. The loss detector uses PLI when `ch->costas` is set and it becomes valid after the first `T_CN0` window.

Signal loss is decided once per `T_CN0` window (not every epoch). A window is flagged bad when the filtered C/N0 falls below the loss threshold (`sdr_thres_cn0_u`, `25 dB-Hz` normally, `32.5 dB-Hz` for L6) **or** the PLI of a Costas-tracked signal falls below `sdr_thres_pli`. A pessimistic counter debounces transients: the channel declares signal loss, returns to idle, and becomes eligible for re-acquisition only after `sdr_lost_th` consecutive bad windows.

4.6 BOC False-Lock Handling

When enabled and the signal uses BOC-like modulation, the tracking bank includes very-early and very-late correlators. Every C/N0 update interval, the receiver compares their accumulated powers with the prompt power. If one side indicates a side-peak lock, the code offset is shifted by a modulation-dependent bump step. This is used to escape false locks on BOC side lobes.

4.7 Tracking State Data

The tracking state is stored in `sdr_trk_t`. Important fields are:

Field	Role
`pos[]`	Correlator positions in samples
`C[]`	Current correlator outputs
`C0`	Previous prompt correlation for FLL
`C1`	Previous epoch's trailing partial correlation, carried for the cross-epoch prompt stitch / bit-transition handling
`P[]`	Prompt-correlation history
`sec_sync`, `sec_pol`	Secondary-code synchronization and polarity

Field	Role
`err_phas`, `phas_acc`	PLL discriminator history and third-order accumulator
`err_code`, `code_int`	DLL discriminator history and second-order integrator
`sumP`, `sumN`	Prompt and noise power sums for C/N0
`sumPs`, `sumD`	Matched prompt-power and `I^2-Q^2` sums for PLI
`Cs`, `coh_n`	Coherent prompt sum and its epoch count for pilot tracking
`sumC[]`, `sumI[]`	DLL accumulation buffers
`code`	Resampled tracking code bank
`code_fft`	Chip-domain extended-code FFT for L6 CSK detection
`csk_ref`	L6 CSK code-shift reference after frame sync (-1: unset)

The channel object `sdr_ch_t` stores the loop state that must persist across epochs: Doppler, code offset, continuous carrier phase (`phi`), accumulated Doppler range, C/N0, carrier lock indicator (`pli`) and its valid flag, week/TOW, lock count, loss count and the loss-decision counter (`lost_cnt`), Costas/non-Costas mode, pilot classification, and navigation state.

`start_track()` resets the dynamic tracking and navigation states. It does not regenerate codes or FFTs because those are permanent channel resources. This makes reacquisition cheap: the channel can move from idle to search to lock without rebuilding signal-specific resources.

4.8 Carrier Wipeoff Implementation

Carrier wipeoff is performed by `sdr_mix_carr()` or fused inside `sdr_corr_std()`. Both accept compact IF samples from an `sdr_buff_t`, a start index, a sample count, a sample rate, and a carrier frequency, and produce `sdr_cpx16_t` samples. `sdr_mix_carr()` writes them to a work buffer for the FFT correlator and the complex-code correlator; `sdr_corr_std()` consumes them chunk by chunk without materializing the full buffer.

The phase passed into `sdr_mix_carr()` is expressed in cycles. The low-level implementation converts phase and frequency step to a fixed-point index into a precomputed complex mixing table. This avoids calling trigonometric functions inside the sample loop.

The tracking loop computes:

$$\tau = t - t_{\text{prev}}$$
$$f_c = f_i + f_d$$
$$\text{adr} \leftarrow \text{adr} + f_d\,\tau$$
$$\phi \leftarrow \phi + f_c\,\tau \quad \text{(wrapped to } [0, 1)\text{)}$$

The mixing carrier phase `phi` (`ch->phi`) is a continuous accumulator of the full carrier `f_c = f_i + f_d`, reduced to `[0, 1)` cycle each epoch by subtracting its floor (this keeps precision bounded; the

mixer only needs the fractional phase). It is kept separate from `adr`, the Doppler-only accumulator used for the carrier-phase observable.

Accumulating `phi` continuously (rather than recomputing `f_i*tau + adr` per epoch) keeps the carrier coherent across code-period boundaries. This matters because the prompt correlation is reconstructed across epochs (the current epoch's leading partial correlation is combined with the previous epoch's trailing partial correlation, carried in the `C1` boundary buffer). Any per-epoch reset of the IF-carrier phase would inject a phase step of `frac(f_i*T)` into that combination. For most signals `f_i*T` is integer (or zero IF) so the step is harmless, but for GLONASS FDMA odd frequency channels `f_i*T = k*562.5` is a half-integer, giving a half-cycle (180°) step that cancels the stitched prompt — previously this appeared as a per-code-cycle code inversion and was worked around with a length-2 secondary code; continuous `phi` removes it at the source.

4.9 Carrier-Aided Code Update

Before correlation, the tracking loop applies carrier aiding to the code offset:

$$\text{coff} \leftarrow \text{coff} - \frac{f_d}{f_{\text{RF}}}\,\tau$$

where `fc_RF` is the RF carrier frequency stored in `ch->fc`. This reflects the fact that satellite-receiver range rate changes both carrier phase and code delay. Carrier aiding reduces the burden on the DLL because the higher-SNR carrier tracking loop supplies most of the short-term dynamics.

The sign convention follows the receiver's Doppler and code-offset definitions. After the carrier-aided update, `adj_coff()` wraps `coff` into `[0, T)` and updates the channel lock count and TOW if a code-period boundary was crossed. The prompt-correlation history is shifted consistently so secondary-code and navigation-symbol timing remain aligned.

4.10 Standard Correlator Details

For ordinary signals, `sdr_corr_std()` wipes off the carrier and forms correlations at the requested positions in one fused pass: each cache-sized chunk of IF samples is mixed and immediately accumulated into all correlator positions. For each position, it splits the local code into two contiguous segments around the code wrap point:

$$C_1 = \text{dot}(IQ[0:j],\ code[N-j:N])$$
$$C_2 = \text{dot}(IQ[j:N],\ code[0:N-j])$$

The dot product is repeated for prompt, early, late, noise, and optional BOC-monitor positions. The function then checks whether the two halves have opposite polarity by evaluating the combined early/prompt/late dot product. If the sign indicates a bit transition, it subtracts the second half instead of adding it.

This is a pragmatic bit-transition mitigation method. It does not decode the data bit during tracking, but it prevents one transition inside the code period from canceling the full coherent integration. The method

is most useful for signals whose data bit or symbol boundary is not yet known.

The standard real-code dot product exploits the local code convention that I and Q code signs are the same. The complex-code version, `sdr_corr_std_cpx_code()`, is used when the local replica itself has complex subcarrier structure, as in the E5 AltBOC path.

4.11 Prompt History and Navigation Timing

After each tracking epoch, the prompt correlation is appended to `trk->P`, a fixed-length history buffer. Several later algorithms read from this history:

- secondary-code synchronization uses a window of recent prompt I values;
- navigation symbol synchronization averages prompt I over a symbol interval;
- navigation decoders collect soft binary symbols into `nav->syms`;
- C/N0 estimation uses prompt power accumulation;
- observation generation uses current tracking state and synchronization flags.

Because the prompt history is indexed by code-period count, the lock counter is part of the timing model. If `coff` wraps and `lock` is adjusted, the history is shifted to keep the apparent prompt-time sequence consistent. This is a small but important detail for signals with secondary codes or long navigation symbol periods.

4.12 FLL Discriminator

The FLL compares the current prompt correlation `P1 = IP1 + j QP1` with the previous prompt correlation `P2 = IP2 + j QP2`. The implementation computes:

$$\begin{aligned}\text{dot} &= I_{P1}I_{P2} + Q_{P1}Q_{P2} \\ \text{cross} &= I_{P1}Q_{P2} - Q_{P1}I_{P2}\end{aligned}$$

For Costas operation, the frequency error uses `atan(cross / dot)`. For non-Costas operation, it uses `atan2(cross, dot)`. The Costas form removes data bit sign ambiguity but has a reduced phase range. The non-Costas form preserves the full phase relationship where data polarity is not being suppressed.

The loop bandwidth is wide during early pull-in and narrow after the initial transient:

$$B = \begin{cases} B_{\text{FLL,w}}, & \text{during early pull-in} \\ B_{\text{FLL,n}}, & \text{after the early period} \end{cases}$$

The Doppler update is proportional to the estimated phase rotation per code period. This pulls the channel close enough for the PLL to take over after the configured pull-in time.

4.13 PLL Discriminator and Loop Filter

The PLL uses the prompt phase:

$$e_\phi = \begin{cases} \dfrac{\tan^{-1}(Q_P/I_P)}{2\pi}, & \text{Costas mode} \\ \dfrac{\operatorname{atan2}(Q_P, I_P)}{2\pi}, & \text{non-Costas mode} \end{cases}$$

The implementation uses a third-order loop with constants documented in the source:

$$W = \frac{B_{\text{PLL}}}{0.7845}$$
$$\Delta t = \text{PLL update interval}$$
$$a_\phi \leftarrow a_\phi + W^3 e_\phi \Delta t$$
$$f_d \leftarrow f_d + 2.4W(e_\phi - e_{\phi,\text{prev}}) + 1.1W^2 e_\phi \Delta t + a_\phi \Delta t$$

For the normal path, `Delta t = T`. For a dataless pilot identified by `sdr_sig_pilot()`, secondary-code sync makes the wiped prompt polarity stable. The receiver sums

$$C_S = \sum_{i=1}^{K} C_{P,i}, \qquad K = \max(1, \text{round}(t_{\text{coh}}/T))$$

and updates the PLL with `atan2(Im(C_S), Re(C_S))` at `Delta t = K*T`. The supported set is L1CP, L5Q, L5SQ, G2OCP, G3OCP, E1C, E5AQ, E5ABQ, E5BQ, E6C, B1CP, B2AP and I1SP. If secondary-code sync is lost, the channel immediately clears the partial coherent sum and returns to the per-period path. If `sdr_b_pll * K * T >= 0.4`, the implementation also falls back to the per-period Costas path to avoid an unstable update interval.

`sec_pol` resolves the current half-cycle branch for wipeoff; it is not the broadcast overlay sequence itself. Integer carrier ambiguity remains, and a polarity change after re-synchronization can cause a half-cycle discontinuity.

The third-order loop can track constant acceleration in carrier phase better than a simple first-order or second-order loop. This is useful for live receivers with oscillator drift, receiver motion, and satellite dynamics.

The PLL does not directly output carrier phase as an observable. Instead, its Doppler estimate advances `adr`, and `gen_cphas()` later converts the accumulated Doppler range into an RTKLIB carrier-phase observable with signal-specific phase-alignment corrections.

4.14 DLL Discriminator and Loop Filter

The DLL accumulates early and late powers over `N = max(1, sdr_t_dll / T)` code periods. At the update boundary:

$$E = \sqrt{\sum |C_E|^2}$$
$$L = \sqrt{\sum |C_L|^2}$$
$$e_{\text{code}} = \frac{E - L}{E + L} \frac{0.5\,T}{N_{\text{code}}}$$

The error is in seconds. The second-order loop uses:

$$
\begin{aligned}
W &= \frac{B_{\mathrm{DLL}}}{0.53} \\
\Delta t &= T\,N \\
i_{\mathrm{code}} &\leftarrow i_{\mathrm{code}} + W^2 e_{\mathrm{code}}\,\Delta t \\
\mathrm{coff} &\leftarrow \mathrm{coff} - (1.414 W e_{\mathrm{code}} + i_{\mathrm{code}})\Delta t
\end{aligned}
$$

The DLL update interval is longer than the carrier update interval because code tracking has lower precision and benefits from non-coherent averaging. The carrier-aided code update handles fast dynamics every epoch, while the DLL removes residual code-phase bias.

4.15 Lock, Loss, and Thresholds

The receiver has three configurable C/N0 thresholds with different meanings:

- `sdr_thres_cn0_l` is used to start lock after acquisition;
- `sdr_thres_cn0_ext` is used for Doppler-assisted acquisition, subject to the lost-threshold-plus-1-dB floor described in 3.3;
- `sdr_thres_cn0_u` is used to declare loss during tracking.

The loss threshold is lower than the acquisition threshold. This hysteresis is intentional. Once a channel is locked, its carrier and code predictions make it possible to continue tracking at lower C/N0 than would be practical for blind search. For L6, a separate threshold is used because the tracking and CSK correlation behavior differs from ordinary ranging signals.

C/N0 alone measures only received power, so it cannot detect a loss of carrier phase lock while prompt power stays high. The loss test therefore combines two detectors evaluated once per `T_CN0` window: the filtered C/N0 against the loss threshold, and the carrier lock indicator (PLI, see 4.5) against `sdr_thres_pli` for Costas-tracked signals. A window is bad when either fails, and a pessimistic counter requires `sdr_lost_th` consecutive bad windows before loss is declared. This keeps brief fades from forcing a false loss while still catching the high-power, phase-unlocked case that C/N0 misses.

On loss, the channel clears tracking lock, secondary-code sync, navigation frame sync, and reverse-polarity flags. It increments `lost` and returns to idle. The scheduler can then attempt re-acquisition, using the last Doppler if the previous lock was long enough and recent enough.

4.16 L6 CSK Tracking

QZSS L6 signals use code shift keying: the 10230-chip code is cyclically shifted by the 8-bit symbol value each 4 ms symbol. On air, L6D and L6E are time-division multiplexed at twice the chip rate; each code chip consists of two sub-chip slots, with L6D occupying the even slot and L6E the odd slot.

Symbol detection and loop tracking are decoupled:

1. One code period is carrier-wiped into the channel work buffer.

2. `sdr_bin_csk()` accumulates the samples of the signal's TDM slot chip by chip, applying the fractional code offset at the bin boundaries, producing a 10230-chip complex sequence.
3. The sequence is circularly correlated with the code by a fixed-size FFT (`CSK_NFFT = 10800`) against a replica periodically extended by `CSK_WIN = 280` chips on both sides, which yields the exact circular correlation for every shift in the +/-280-chip window without wrap-around aliasing. The peak over the candidate shifts gives the CSK shift, and the symbol is appended to the navigation symbol buffer.
4. The E/P/L/N correlator outputs are produced by the standard time-domain correlator with the detected shift added to the code offset, so FLL, PLL, DLL, and C/N0 operate on true (non-interpolated) correlations of the CSK-shifted code. The standard correlator is called with fixed code polarity (`pol = 1`): the L6 window is symbol-aligned and circular, so no data-bit flip can occur at the code wrap-around, and the normal flip detector would fire randomly whenever one partial correlation is noise-dominated.

Because the correlation FFT size is fixed, the symbol-detection cost is independent of the sampling rate; only carrier wipeoff, chip binning, and the standard correlator scale with `fs`.

Before frame synchronization, the code-phase reference established at acquisition is offset by the unknown symbol transmitted during acquisition, so the peak search covers shifts in [-255, +255]. This window contains each symbol value twice: shifts `s` and `s - 256` decode to the same symbol, but only one of them places the loop correlators on the true peak. Once the L6 navigation decoder achieves frame sync and recovers the constant symbol offset (stored in `nav->coff`), the channel anchors `trk->csk_ref` and narrows the search to the 256 shifts `[csk_ref - 255, csk_ref]`, eliminating the alias candidates. The anchor is cleared when frame sync is lost, so a wrong anchor self-heals within one frame.

4.17 E5 AltBOC Tracking

The `E5ABQ` implementation is a sign-only, pilot-only AltBOC approximation. It uses an E5aQ replica for acquisition. During tracking it uses one of three banks:

Bank	Use
0	E5aQ-only tracking before secondary-code sync
1	E5aQ/E5bQ combination for one relative secondary-code polarity
2	E5aQ/E5bQ combination for the opposite relative secondary-code polarity

The bank is selected after E5aQ secondary-code sync from the relative polarity between E5aQ and E5bQ secondary codes. The implementation also supports an E5b-E5a group-delay offset option, applied when generating the E5bQ bank. The resulting complex-code correlator lets the existing tracking loop handle E5 AltBOC without a separate tracking state machine.

4.18 Tracking Pseudocode

The normal tracking path can be summarized as:

```
tau = time - ch.time
fc = ch.fi + ch.fd
ch.adr += ch.fd * tau
ch.coff -= ch.fd / ch.fc * tau
ch.phi += fc * tau; ch.phi -= floor(ch.phi)   # continuous carrier phase [0,1)
ch.time = time
wrap code offset if needed

mix carrier from IF buffer
correlate prompt, early, late, noise, optional monitor taps
append prompt correlation to history
update TOW by one code period if valid
lock += 1

if secondary code can be synchronized:
    sync and wipe secondary code

if inside pull-in time:
    run FLL
else if synchronized supported pilot and b_pll * K * T < 0.4:
    sum K secondary-code-wiped prompts
    run atan2 PLL every K periods with dt = K * T
else:
    run per-period Costas/non-Costas PLL with dt = T

run DLL
update noise-subtracted C/N0 and matched-interval PLI

if navigation pull-in reached:
    decode navigation data

once per C/N0 window:
    if C/N0 OR carrier-lock (PLI) is bad for sdr_lost_th windows:
        declare signal lost
```

This ordering matters. For example, secondary-code wipeoff occurs before loop updates and navigation decoding, so the loop discriminators and symbol generation see polarity-corrected prompt values once secondary sync is known.

5. Navigation Data Decoding

Navigation decoding is run from `track_sig()` after navigation pull-in. The dispatcher `sdr_nav_decode()` selects a signal-specific decoder based on `ch->sig`.

The common decoder pattern is:

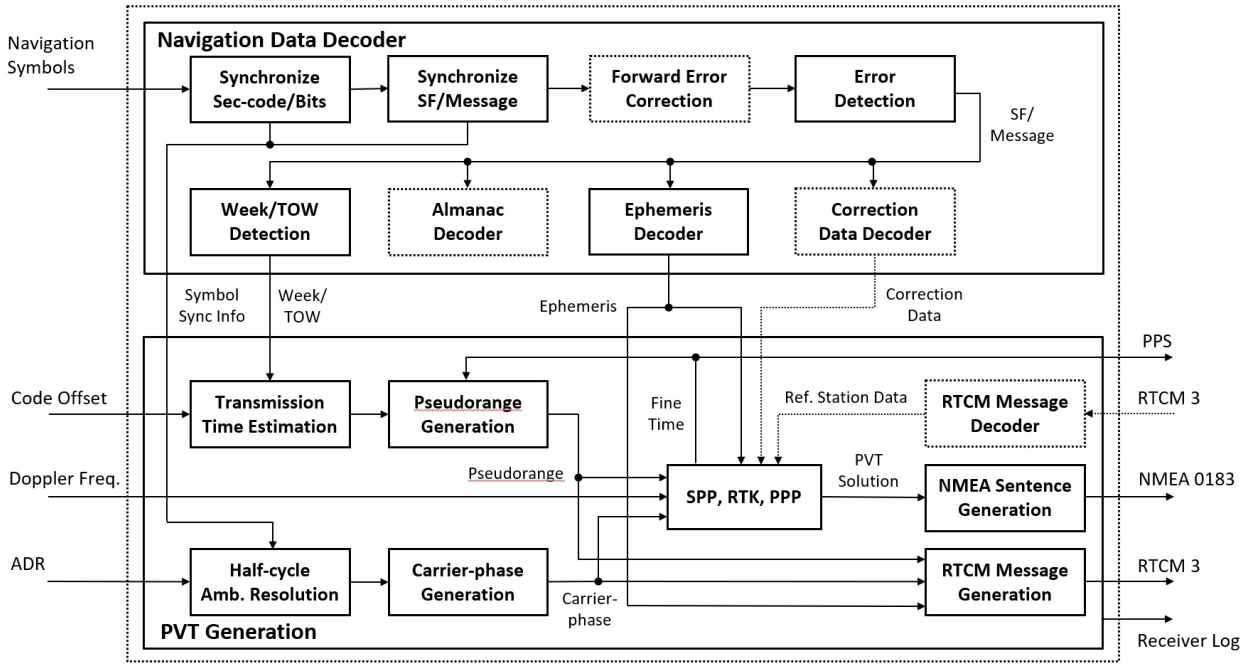
1. Convert prompt I correlations to soft binary symbols, or collect one soft symbol per secondary-code epoch.
2. Establish symbol sync by checking bit transitions or secondary-code timing.
3. Search frame preambles or known synchronization symbol patterns.
4. Decode FEC if the signal uses convolutional, LDPC, BCH, or other coding.
5. Check parity or CRC.
6. Update week/TOW, frame type, raw navigation data buffer, and status flags.
7. Emit `$NAV` log records and mark `ch->nav->stat = 1` for PVT ingestion.

Examples:

- GPS/QZSS `L1CA` uses 20 ms symbol sync, LNAV preamble detection, and LNAV parity over 10 words.
- GPS/QZSS `L1CD` decodes CNAV-2 by deinterleaving subframes, LDPC decoding, and CRC checks.
- GPS/QZSS `L2CM` and `L5I` use CNAV frame sync, convolutional decoding, and CRC24Q.
- Pilot-only signals such as `L1CP` establish timing from secondary-code sync and set unresolved time ambiguity flags instead of decoding a data frame.
- Galileo, GLONASS, BeiDou, NavIC, QZSS L6, and SBAS paths each have dedicated frame/message decoders and consistency checks.

When a frame is decoded, the channel stores packed raw navigation data in `ch->nav->data`, sets `ch->nav->type`, updates week/TOW where available, and increments decode counters. If frame sync or parity/CRC fails, the decoder clears navigation sync and TOW validity.

The figure below summarizes the navigation data decoder and the downstream PVT generation covered in §6.



PVT: position, velocity and timing, SF: subframe, SPP: single point positioning, RTK: Real-time kinematic, PPP: Precise point positioning

Block Diagram of Navigation Data Decoder and PVT Generation

5.1 Navigation Decoder State

Navigation decoder state is stored in `sdr_nav_t`:

Field	Meaning
`ssync`	Symbol synchronization time as lock count
`fsync`	Frame synchronization time as lock count
`rev`	Reversed polarity flag
`nerr`	Number of corrected FEC errors for the last frame
`seq`	Frame or page sequence value
`type`	Message type, subframe ID, page type, or signal-specific type
`stat`	New navigation data is available for PVT ingestion
`coff`	Extra code offset used by some ambiguous timing paths
`syms[]`	Rolling soft-symbol buffer for binary navigation symbols; L6 stores CSK symbols
`data[]`	Packed raw decoded navigation frame/message
`lock_sf[]`	Lock times of recently decoded subframes
`count[]`	Successful and failed decode counters

`sdr_nav_init()` resets this state when tracking starts. If a decoder detects a loss of symbol/frame consistency, `unsync_nav()` clears symbol sync, frame sync, polarity, code-offset helper state, and TOW

validity. This conservative reset prevents stale navigation time from being used for observations.

5.2 Symbol Generation

Most navigation decoders use prompt I correlations. The helper `IP2sym()` converts the latest prompt I value to a soft binary symbol through `corr2soft()`:

$$\text{soft} = \begin{cases} 0, & 128 - kI_P \le 0 \\ 255, & 128 - kI_P \ge 255 \\ \text{round}(128 - kI_P), & \text{otherwise} \end{cases}$$

where `k` is the implementation scale factor. The convention is:

- `0` means a strong symbol 0;
- `128` is the hard-decision boundary and lowest confidence point;
- `255` means a strong symbol 1.

Hard-decision processing uses `hard_sym()`, which maps values below 128 to 0 and values of 128 or greater to 1. Polarity reversal preserves reliability by using `255 - soft` before hard decisions or FEC decoding.

For signals with known symbol intervals, `sync_symb()` averages prompt I over `N` code periods and appends one soft symbol when a symbol boundary is reached. The initial symbol sync test keeps the conservative hard-compatible transition check: recent candidate symbol blocks must have stable prompt-I signs and the two latest blocks must show a sign transition. Once synchronized, the appended symbol keeps the averaged prompt-I reliability as a soft value. If the average prompt magnitude falls below the symbol-loss threshold, symbol sync is cleared.

For pilot or secondary-code-timed signals, `sync_sec_code()` appends one symbol per secondary-code period after tracking secondary-code sync is available. The stored value is also soft, based on the averaged prompt I over the secondary code period. This allows the navigation decoder to use overlay-code timing rather than searching for data bit transitions.

L6 is different because CSK symbols are produced by the tracking correlator, not by prompt I sign. `CSK()` appends the detected CSK symbol number to `nav->syms`, and the L6 navigation decoders operate on that symbol stream. This is the main exception to the binary soft-symbol convention for `nav->syms`.

5.3 Frame Synchronization

The generic `sync_frame()` helper searches for two instances of a known preamble separated by the frame length. It checks both normal and reversed polarity:

```
normal:   preamble matches bits[0:n] and bits[N:N+n]
reversed: inverted preamble matches both positions
```

The returned polarity is stored in `nav->rev` after a successful decode. Later frames are expected to maintain the same polarity. If the expected next frame does not appear at the predicted lock count, the decoder clears frame sync.

For frame synchronization over raw navigation-symbol buffers, the decoder uses soft-symbol score helpers for preambles and known symbol tables. Each expected bit is scored from the stored reliability value, and the acceptance threshold is chosen so that saturated 0/255 inputs keep the same allowed-error behavior as the previous hard-symbol matcher. This lets low-confidence sign errors pass when the rest of the sync pattern is reliable, while strong wrong symbols still reject the candidate. Patterns configured with zero allowed errors remain hard-compatible and require exact hard-decision agreement to avoid false locks on short preambles.

Modern signals often do not use a simple repeated preamble. The implementation therefore has signal-specific synchronizers for CNAV-2, Galileo, BeiDou B-CNAV, GLONASS modernized strings, and NavIC L1 SPS. These synchronizers use known subframe symbol tables, preamble-like sync words, page sequence fields, or CRC success depending on the signal definition.

5.4 FEC and Integrity Checks

The navigation decoder uses multiple integrity mechanisms:

- LNAV parity for GPS/QZSS L1 C/A;
- CRC24Q for CNAV-like messages and several modernized frames;
- GLONASS CRC16 for modernized strings;
- soft-input convolutional decoding for CNAV and related messages;
- LDPC decoding for CNAV-2, Galileo CNAV/E6, BeiDou modernized signals, and NavIC L1 SPS where implemented;
- BCH correction for BeiDou D1/D2 substructures.

The convolutional decoder, `sdr_decode_LDPC()`, and `sdr_decode_NB_LDPC()` accept `uint8_t` soft symbols in the range 0 to 255. Hard-decision operation remains available by passing only 0 and 255 values. Parity-only, BCH, and CRC validation paths use hard bits derived locally from the soft symbols.

The decoder does not pass a message to PVT merely because a preamble was found. It sets `nav->stat` only after the message passes its parity/CRC/FEC criteria and the signal-specific timing fields are extracted. This protects the PVT layer from inconsistent ephemeris and TOW updates.

5.5 GPS and QZSS Paths

GPS/QZSS `L1CA` and QZSS `L1CB` use the LNAV path. After 20 ms symbol sync, the decoder searches for the 8-bit LNAV preamble and validates the 10-word subframe parity. It extracts subframe ID, week number from subframe 1, and TOW from the HOW word. The decoded 24-bit data words are packed into `nav->data`.

`L2CM`, `L5I`, and related CNAV-bearing signals use convolutional decoding and CRC24Q. The receiver collects enough symbols for a CNAV frame, decodes the rate-1/2 convolutional code, searches for the CNAV preamble, checks CRC, and extracts message type and TOW. `L5Q` is a pilot component and uses timing support rather than a normal data frame.

`L1CD` uses CNAV-2. The decoder uses a known subframe-1 symbol table to synchronize the frame and time-of-interval value. It deinterleaves the symbol matrix, decodes LDPC blocks for subframes 2 and 3, checks CRCs, and packs SF1, SF2, and SF3 data into the channel navigation buffer. `L1CP` is the matching pilot path and obtains ambiguous timing from secondary-code sync.

QZSS L6 `L6D` and `L6E` are decoded from CSK symbols. The tracking path detects the CSK shift and appends symbols; the navigation decoder then synchronizes the frame on preamble differences, recovers the constant symbol offset, and checks the Reed-Solomon code. After frame sync, the recovered symbol offset (`nav->coff`) is also fed back to tracking to narrow the CSK peak search (see 4.16).

5.6 GLONASS Paths

Legacy GLONASS `G1CA` and `G2CA` decode GLONASS navigation strings after symbol sync and frame search. PVT ingestion stores GLONASS ephemerides in `geph[]` and records the frequency channel number from `ch->prn`. For FDMA signals this FCN is essential because satellite frequency affects measurement modeling.

Modernized GLONASS data-bearing paths implemented in the navigation decoder, such as `G1OCD` and `G3OCD`, use signal-specific string and CRC handling. `G3OCP` provides pilot timing from secondary-code sync. `G1OCP` and `G2OCP` can be acquired and tracked, but their navigation-decoder paths are not implemented in `sdr_nav_decode()`. The PVT layer currently ingests ephemerides from the implemented data-bearing paths and uses pilot tracking primarily for observables and timing support.

5.7 Galileo Paths

Galileo `E1B` and `E5BI` use I/NAV decoding. `E5AI` uses F/NAV decoding. Galileo `E1C`, `E5AQ`, `E5BQ`, `E5ABQ`, and `E6C` include pilot or modernized paths that depend on secondary-code sync and signal-specific frame structures.

The PVT ingestion layer distinguishes Galileo I/NAV and F/NAV ephemerides by where they are stored in the RTKLIB navigation object. `E1B` / `E5BI` update the normal Galileo ephemeris slot, while `E5AI` updates the alternate slot used for F/NAV. This allows the receiver to keep both navigation sources when available.

For `E5ABQ`, tracking uses an AltBOC pilot combination, but navigation decoding is mapped to the `E5AQ` path. The channel therefore benefits from wideband pilot tracking while reusing existing Galileo E5aQ timing behavior.

5.8 BeiDou Paths

BeiDou legacy `B1I`, `B2I`, and `B3I` decode D1 or D2 navigation depending on PRN. The PVT ingestion path applies an additional consistency check for D1/D2 ephemerides: it compares a newly decoded ephemeris with the previous candidate before accepting it as current. This reduces the chance of using a corrupted message that happened to pass lower-level checks.

BeiDou modernized signals `B1CD`, `B1CP`, `B2AD`, `B2AP`, and `B2BI` have dedicated B-CNAV decoders and symbol/frame synchronizers. Pilot channels provide tracking and ambiguous timing, while data channels provide ephemeris and clock information.

5.9 NavIC and SBAS Paths

NavIC `I5S` and `ISS` use NavIC navigation decoding and update the RTKLIB ephemeris object when a valid message is decoded. `I1SD` and `I1SP` represent L1 SPS data and pilot components, using signal-specific synchronization tables and decoding paths.

SBAS is handled through `L1CA`, `L5I`, or `L5Q` signal IDs with SBAS PRN ranges. For SBAS messages, the receiver outputs raw navigation logs and, where the message type is a GEO navigation message, updates the RTKLIB SBAS correction state.

5.10 Timing Validity

The navigation decoder controls whether observations can be generated. The important channel fields are:

- `week`: GNSS week number if decoded;
- `tow`: time of week in milliseconds;
- `tow_v`: timing validity flag;
- `nav->fsync`: frame synchronization lock count;
- `trk->sec_sync`: secondary-code synchronization lock count.

`tow_v = 1` means that TOW has been validated by consistent decoded message timing. `tow_v = 2` means timing is ambiguous but may be resolvable, for example from a companion observation. `tow_v = 0` means the channel must not produce an absolute pseudorange. This separation lets pilot tracking contribute when it can be tied to a decoded time source without pretending that an unresolved time is absolute.

6. Observation and PVT Generation

6.1 Navigation Data Ingestion

After each channel update, the channel thread checks `ch->nav->stat`. If it is set, `sdr_pvt_udnav()` transfers decoded navigation data to the shared RTKLIB navigation object. Depending on signal type, it decodes and stores:

- GPS/QZSS LNAV ephemeris and ionospheric parameters;
- Galileo I/NAV and F/NAV ephemerides;
- GLONASS ephemerides;
- BeiDou D1/D2 ephemerides, with a repeated-ephemeris consistency check;
- NavIC ephemerides;
- SBAS correction messages;
- RTCM3 navigation messages for output where applicable.

6.2 Observation Generation

Observation epochs are initialized from a channel with valid week/TOW. The epoch cycle is aligned to the configured observation interval and rounded by 20 ms.

For each locked channel at the epoch, `sdr_pvt_udobs()` generates observations only if:

- TOW is available and valid or ambiguity-resolvable;
- frame sync or secondary-code sync is available;
- code offset and satellite ID are valid.

Pseudorange is computed from receiver epoch time, decoded channel time, and tracked code offset. For channels with only ambiguous timing, a 100 ms ambiguity resolution path is used when supported. Carrier phase is derived from accumulated Doppler range and corrected for known half-cycle, secondary-code, and signal-specific phase alignments. Doppler and C/N0 are copied from tracking state.

The generated RTKLIB observation fields are:

- `P`: pseudorange;
- `L`: carrier phase;
- `D`: Doppler;
- `SNR`: C/N0 converted to RTKLIB SNR units;
- `LLI`: loss-of-lock and half-cycle ambiguity indicators;
- `code`: RINEX observation code selected from the signal ID.

Before solution generation, the receiver resolves selected millisecond ambiguities against companion observations, for example GPS/QZSS L5Q against another signal and GLONASS G3OCP against another

GLONASS observation.

6.3 PVT Solution

At each epoch, observations are sorted and output as logs and RTCM3 observation messages. The PVT solver then runs RTKLIB single-point positioning using L1 pseudoranges, broadcast ephemerides, broadcast ionosphere, Saastamoinen troposphere, elevation mask, and RAIM-FDE enabled.

If positioning succeeds, the receiver:

- corrects solution time when the primary GPS clock offset is absent but another system clock offset is available;
- outputs `$POS`, NMEA RMC/GGA/GSV, and `$SAT` logs;
- updates solution counters and displayed status.

If positioning fails, satellite azimuth/elevation is still updated from available ephemerides for status display.

After each epoch the receiver advances to the next observation time, clears the observation buffer, records solution latency, and optionally adjusts the next epoch cycle by the estimated receiver clock offset.

6.4 PVT Object State

The `sdr_pvt_t` object is the bridge between channel-level tracking and receiver-level navigation output. It contains:

Field	Role
`time`	Current observation epoch time
`ix`	Receiver cycle index for the current epoch
`nsat`	Number of satellites in the current status view
`nch`	Number of channels that have reported for the current epoch
`obs`	RTKLIB observation buffer
`nav`	RTKLIB navigation data buffer
`sol`	RTKLIB solution object
`ssat`	Per-satellite status
`rtcm`	RTCM encoder control
`latency`	Difference between input cycle and solution epoch
`count[]`	Solution, observation, and navigation counters

The PVT object is shared by all channel threads and the receiver thread. It is therefore always accessed under `pvt->mtx`. Channel threads add navigation data and observations; the receiver thread finalizes epochs and runs PVT.

6.5 Epoch Initialization

The PVT epoch cannot be initialized until at least one channel has decoded or otherwise established GNSS time. `init_epoch()` uses a channel's week and TOW to set `pvt->time` and `pvt->ix`. The cycle index is rounded to a 20 ms boundary:

$$i_{\text{pvt}} = i_x + \text{round}(\frac{t_{\text{epoch}} - t_{\text{ch}} - 0.07}{T_{\text{cyc}}})$$

$$i_{\text{pvt}} \leftarrow 20\lfloor \frac{i_{\text{pvt}}}{20} \rfloor$$

The small offset and 20 ms rounding reflect the receiver's observation epoch alignment strategy. Once the epoch is initialized, subsequent PVT updates advance by `sdr_epoch`.

This design means that the receiver can begin tracking before it can produce observations or PVT. It first locks channels, then decodes time, then initializes the PVT epoch, then starts generating synchronized observations.

6.6 Pseudorange Model

For a channel with decoded week and TOW, pseudorange is generated as:

$$\begin{aligned} t_{\text{rcv}} &= \text{receiver epoch time in GNSS time scale} \\ t_{\text{sig}} &= \text{channel decoded transmit-time marker} \\ \tau &= t_{\text{rcv}} - t_{\text{sig}} + \text{coff} \\ P &= c\,\tau \end{aligned}$$

The implementation accounts for week rollover between the PVT epoch and the channel's decoded week. The code offset `coff` is added because the channel TOW refers to a code/frame boundary, while the prompt tracking point is offset inside the current code period.

For ambiguous timing, the implementation first folds the time difference into a 0.05 to 0.15 s interval when the channel marks `tow_v = 2` and no decoded week is available. Some signal paths add an extra helper offset before this folding. Selected short-period ambiguous observations are then refined or invalidated by the companion-observation resolver described in §6.10.

If pseudorange cannot be generated reliably, `update_obs()` returns without adding a measurement. The receiver prefers missing observations over observations with stale or ambiguous time.

6.7 Carrier Phase Model

Carrier phase is generated primarily from accumulated Doppler:

$$L = -\text{adr}$$

Additional half-cycle and quarter-cycle corrections are then applied:

- navigation polarity reversal can add 0.5 cycle;
- secondary-code polarity can add 0.5 cycle;
- several signals require fixed quarter-cycle phase alignment corrections;
- some BeiDou GEO/IGSO/MEO legacy paths require additional half-cycle handling.

The result is a carrier-phase observable in cycles. It is not ambiguity-fixed; it is a continuous tracking observable with possible loss-of-lock indicators. The PVT solver in this receiver path uses pseudorange for single-point positioning, but carrier phase is still logged and emitted in observation streams for external processing.

6.8 Doppler and SNR Observables

Doppler is stored directly from the channel's tracked `fd`. The sign convention is the internal receiver convention used consistently by tracking and observation output. C/N0 is converted to RTKLIB SNR units:

$$\mathrm{SNR} = \mathrm{round}\left(\frac{C/N_0}{\mathrm{SNR\_UNIT}}\right)$$

The observation also carries loss-of-lock indicators. The implementation sets the PLL-unlock bit when the lock time is short or the phase error is too large. It sets the half-cycle ambiguity bit when neither frame sync nor secondary-code sync is known.

6.9 Observation Indexing

Each channel has an `obs_idx` assigned by the PVT object. This maps a signal ID to the observation code slot used by RTKLIB. Multiple signals from the same satellite can therefore appear in one observation record with different code indices. For array or per-RF-channel observations, the receiver also uses the RTKLIB receiver number field to distinguish channels where needed.

The observation update avoids duplicate satellite records by searching the current epoch buffer for the same satellite and, for some modes, the same receiver number. If no record exists and the buffer has capacity, it creates a new `obsd_t` entry and fills only the fields for the current signal.

6.10 Millisecond Ambiguity Resolution

Some pilot or long-code observations provide pseudorange modulo a short period. Before solving PVT, `sdr_pvt_udsol()` calls `res_obs_amb()` for selected signal families:

- GPS/QZSS `L5Q` against another GPS/QZSS observation with a 20 ms period;
- QZSS `L5SQ` / `L5SQV` with a 20 ms period;
- GLONASS `G3OCP` with a 10 ms period;
- SBAS `L5Q` with a 2 ms period.

The resolver uses a companion observation from the same satellite. It replaces the ambiguous pseudorange with the nearest value consistent with the companion range. If no companion is available, the ambiguous pseudorange is invalidated.

This policy is conservative. It allows pilot observations to contribute when another signal provides the coarse range, but prevents standalone ambiguous pilot measurements from corrupting the PVT solution.

6.11 Navigation Data Update Details

`sdr_pvt_udnav()` translates channel navigation frames into RTKLIB navigation objects. It uses signal ID and satellite system to choose the decoder:

- GPS/QZSS LNAV is decoded by RTKLIB `decode_frame()`;
- GLONASS legacy navigation is decoded by `decode_glostr()`;
- Galileo I/NAV and F/NAV use `decode_gal_inav()` and `decode_gal_fnav()`;
- BeiDou D1/D2 use `decode_bds_d1()` and `decode_bds_d2()`;
- NavIC uses `decode_irn_nav()`;
- SBAS messages update SBAS correction state when applicable.

When a valid ephemeris is accepted, the receiver logs `$EPH`-style information through `out_log_eph()` and can emit RTCM3 navigation messages through the RTCM stream. The navigation counter `pvt->count[2]` is incremented only for accepted updates.

6.12 Point Positioning Configuration

The point-positioning call uses RTKLIB's `pntpos()` with a receiver-local option configuration:

- navigation systems include GPS, GLONASS, Galileo, QZSS, BeiDou, and NavIC;
- broadcast ephemerides are used;
- broadcast ionosphere correction is enabled;
- Saastamoinen troposphere correction is enabled;
- elevation mask comes from the receiver option `sdr_el_mask`;
- RAIM-FDE is enabled.

Before calling `pntpos()`, the receiver selects one L1-like pseudorange per satellite. The implementation deletes duplicated L1 observations by keeping the first valid pseudorange for each satellite. This keeps the single-point solver input simple even though the observation buffer may contain multiple signals per satellite for logging or external users.

6.13 Output Products

The receiver path can output several product streams:

- status text for console/UI display;
- `$CH`, `$OBS`, `$NAV`, `$POS`, `$SAT`, and other log records;
- RTCM3 MSM observation messages;
- RTCM3 navigation messages;
- NMEA `RMC`, `GGA`, and `GSV` sentences;
- raw IF data stream and tag file.

These outputs are derived from the same PVT and channel state. The log stream is therefore useful for debugging receiver behavior: acquisition events, signal loss, navigation decode success/failure, ephemeris updates, observation epochs, and point-positioning errors all appear in a common receiver-time sequence.

6.14 Failure Handling in PVT

PVT generation can fail even when tracking is healthy. Common causes are:

- no decoded ephemeris for enough tracked satellites;
- insufficient satellite geometry;
- invalid or unresolved pseudorange timing;
- inconsistent navigation frames;
- RAIM-FDE rejection;
- elevation mask excluding too many satellites.

When `pntpos()` fails, the receiver does not clear tracking channels. It logs a positioning error, updates satellite azimuth/elevation from available ephemerides, sets the solution satellite count to zero, and waits for the next epoch. This separation is important: tracking is a signal-processing state, and PVT validity is a navigation-solution state.

7. Implementation for Speed

The receiver is written around several practical speed choices.

7.1 Fixed 1 ms Input Cycle and Circular Buffers

Input is processed in 1 ms blocks. Each RF channel has a circular buffer with up to `MAX_BUFF` cycles. Channel threads consume from these buffers independently, so acquisition and tracking can lag the input stream without stopping RF input immediately. Buffer usage is monitored, and new searches are suppressed when the usage exceeds the configured high-water mark.

7.2 Lookup Tables for Sample Conversion

Raw FE formats are unpacked with lookup tables instead of per-sample arithmetic. Separate LUTs handle:

- packed 2-bit and 3-bit FE raw samples;
- signed 8-bit and 16-bit complex input normalization;
- `fs/4` real-to-complex mixing for real-sampled channels.

For generic integer formats, the receiver estimates input mean and standard deviation and periodically regenerates scaling LUTs to target the internal sample range.

7.3 Precomputed Code Banks

Acquisition code FFTs are generated once per channel. Tracking replicas are also precomputed at `SDR_N_CODES` fractional code phases, so per-epoch tracking only selects the nearest fractional bank and runs dot products. This avoids resampling the spreading code during tracking.

7.4 FFT and SIMD Correlators

Acquisition uses FFT correlation over all code phases for each Doppler bin. Tracking uses time-domain dot products for normal short-code tracking, which is cheaper than FFT when only a few correlator taps are needed. L6 additionally runs a small fixed-size chip-domain FFT each symbol to search the CSK code-phase shift; its E/P/L correlations use the time-domain correlator like any other signal.

The low-level carrier mixer and correlator include AVX2 and NEON paths where available. Complex samples and code replicas are stored in compact integer formats so the hot loops can use vector integer operations.

7.5 Search Load Control and Doppler Assistance

Only one idle channel is actively acquired at a time. This prevents acquisition from starving tracking. Re-acquisition and same-satellite Doppler assistance reduce the number of Doppler bins, often to three bins,

which greatly reduces the FFT search cost for reacquired or companion signals.

7.6 Multi-Threaded Channel Tracking

Each receiver channel has its own thread. Locked channels advance independently as soon as enough IF data is present. Shared receiver and PVT state is protected by mutexes around buffer index, channel tracking state, and PVT updates.

7.7 Memory Layout Choices

The hot-path memory layout is chosen to reduce copies and cache pressure:

- raw input is read into a temporary byte buffer for one 1 ms receiver cycle;
- unpacked IF samples are written directly into RF-channel circular buffers;
- carrier-wiped samples are consumed chunk by chunk inside the standard correlator; only the L6 CSK and complex-code paths produce them into a channel-local work buffer;
- code replicas are precomputed in channel-local contiguous banks;
- correlator outputs are small fixed arrays in the tracking state;
- prompt history is a fixed rolling array rather than a dynamically growing buffer.

The channel-local design avoids sharing large working buffers between threads. It also makes the lifetime of memory clear: code banks live for the channel lifetime, acquisition sums live for one acquisition attempt, and carrier-wiped data lives for one tracking/acquisition operation.

7.8 Integer Sample Scaling

The internal IF representation uses compact signed integer values. For generic input formats, the receiver estimates mean and standard deviation from a small sample subset and regenerates LUTs so that the output values have a target standard deviation. The default target is controlled by `AGC_LEVEL`.

This is not an RF AGC loop. It does not change front-end gain. It is a digital normalization step that keeps the integer correlator inputs in a useful range. For SoapySDR devices or external front-ends, it helps absorb differences in driver sample scaling. For Pocket SDR packed raw formats, the quantization levels are already known, so the packed-sample LUTs are fixed by bit width and sampling mode.

7.9 FFTW and FFT Abstraction

The FFT helper `sdr_cpx_fft()` hides the FFT implementation. Acquisition uses it through `sdr_corr_fft()`, and L6 CSK tracking uses it directly for the fixed-size chip-domain correlation. The codebase also supports FFTW wisdom generation and loading in the wider Pocket SDR toolchain. In the receiver path, the main algorithmic point is that code FFTs are generated once, while data FFTs are generated for each Doppler bin or each L6 symbol.

The PCPS acquisition complexity for one coherent integration is roughly:

$$O(N_{\text{dop}}(\text{FFT}(2N) + 2N))$$

where `Ndop` is the number of Doppler bins. This is why Doppler assistance and direct-acquisition limits matter. The code-phase dimension is handled efficiently by FFT, but the Doppler dimension still requires one carrier wipeoff and FFT correlation per bin.

7.10 SIMD Hot Paths

The low-level functions include AVX2 and NEON blocks for two key operations:

- carrier mixing from compact IF samples to `sdr_cpx16_t`;
- dot products between carrier-wiped samples and code replicas.

The vector paths exploit the fact that code samples are signs and IF samples are compact integers. Multiplication by a sign can be implemented with sign operations rather than general floating-point multiplication. The scalar loops remain as portable fallbacks.

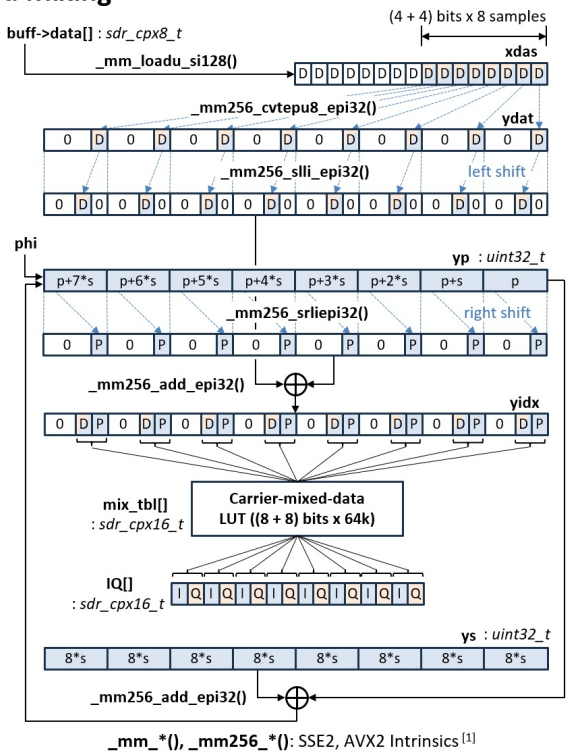
The implementation keeps the public algorithm independent of the SIMD path. Acquisition, tracking, and PVT behavior should not change when AVX2 or NEON is not available; only throughput changes.

The two figures below show the AVX2 vectorization of the two hottest kernels, carrier mixing (`mix_carr()`) and the correlator inner product (`dot_IQ_code()`).

Carrier NCO and Mixing

PocketSDR/src/sdr_func.c (ver.0.13)

```
void mix_carr(const sdr_buff_t *buff, int ix, int N, double phi,
              double step, sdr_cpx16_t *IQ) { // step = Δphi / sample
  const uint8_t *data = buff->data + ix;
  double scale = (double)(1 << 24) * NTBL;
  uint32_t p = (uint32_t)((phi - floor(phi)) * scale); // initial phase
  uint32_t s = (uint32_t)(int)(step * scale); // phase step / sample
  int i = 0;
  __m256i yp = _mm256_set_epi32(p+s*7, p+s*6, p+s*5, p+s*4, ...);
  __m256i ys = _mm256_set1_epi32(s*8);
  for ( ; i < N - 16; i += 8) {
    int idx[8];
    __m128i xdas = _mm_loadu_si128((__m128i *)(data + i));
    __m256i ydat = _mm256_cvtepu8_epi32(xdas);
    __m256i yidx = _mm256_add_epi32(_mm256_slli_epi32(ydat, 8),
           _mm256_srli_epi32(yp, 24));
    _mm256_storeu_si256((__m256i *)idx, yidx);
    IQ[i  ] = mix_tbl[idx[0]]; // IQ(t)=data(t)*exp(-j*2*pi*(f*t+phi))
    IQ[i+1] = mix_tbl[idx[1]];
    IQ[i+2] = mix_tbl[idx[2]];
    ...
    IQ[i+7] = mix_tbl[idx[7]];
    yp = _mm256_add_epi32(yp, ys);
  }
  for (p += s * i; i < N; i++, p += s) {
    int idx = ((int)data[i] << 8) + (p >> 24);
    IQ[i] = mix_tbl[idx];
  }
}
```



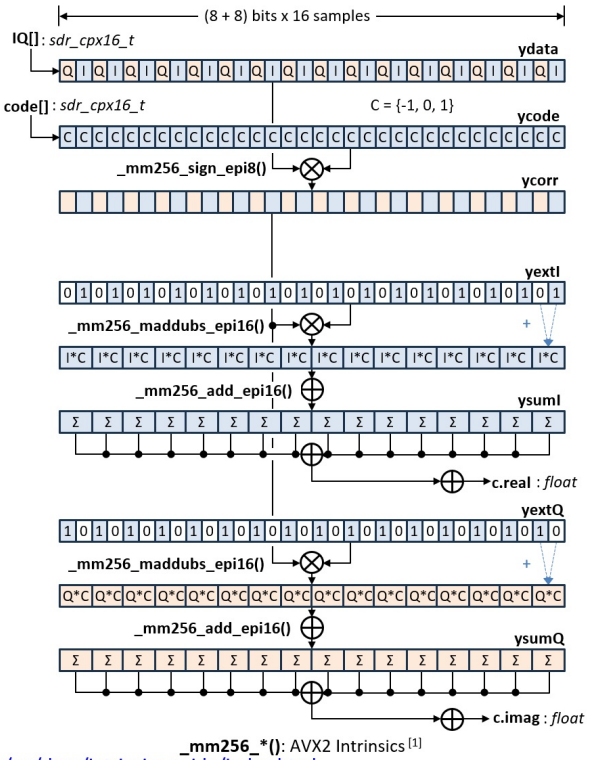
[1] https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

SIMD (AVX2) Optimization of Carrier Mixing

Correlator

PocketSDR/src/sdr_func.c

```
void dot_IQ_code(const sdr_cpx16_t *IQ, const sdr_cpx16_t *code, int N,
                 float s, sdr_cpx_t *c) { // c: correlation
  int i = 0;
  (*c)[0] = (*c)[1] = 0.0f;
  __m256i ysumI = _mm256_setzero_si256();
  __m256i ysumQ = _mm256_setzero_si256();
  __m256i yextI = _mm256_set_epi8(0,1,0,1,0,1,0,1,0,1,0,1,...);
  __m256i yextQ = _mm256_set_epi8(1,0,1,0,1,0,1,0,1,0,1,0,...);
  for ( ; i < N - 15; i += 16) {
    __m256i ydata = _mm256_loadu_si256((__m256i *)(IQ + i));
    __m256i ycode = _mm256_loadu_si256((__m256i *)(code + i));
    __m256i ycorr = _mm256_sign_epi8(ydata, ycode);
    ysumI = _mm256_add_epi16(ysumI, _mm256_maddubs_epi16(yextI, ycorr));
    ysumQ = _mm256_add_epi16(ysumQ, _mm256_maddubs_epi16(yextQ, ycorr));
    if (i % (16 * 256) == 0) { // to avoid overflow
      sum_s16(ysumI, (*c)[0])  // c.real += sum(ysumI), ysumI = 0
      sum_s16(ysumQ, (*c)[1])  // c.imag += sum(ysumQ), ysumQ = 0
    }
  }
  sum_s16(ysumI, (*c)[0]) // c.real += sum(ysumI)
  sum_s16(ysumQ, (*c)[1]) // c.imag += sum(ysumQ)
  for ( ; i < N; i++) {
    (*c)[0] += IQ[i].I * code[i].I;
    (*c)[1] += IQ[i].Q * code[i].Q;
  }
  (*c)[0] *= s * SDR_CSCALE; // correlation I
  (*c)[1] *= s * SDR_CSCALE; // correlation Q
}
```



[1] https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

SIMD (AVX2) Optimization of Correlator

7.11 Scheduler as a Real-Time Protection Mechanism

The receiver scheduler is a speed feature, but it is also a reliability feature. Acquisition can be much more expensive than tracking because it searches many Doppler bins and all code phases. If every idle channel were allowed to search at once, the input circular buffers could overflow and locked channels could fall behind.

`update_srch_ch()` therefore enforces:

- no new acquisition when buffer usage is too high;
- no new acquisition while another channel is already in `SDR_STATE_SRCH`;
- round-robin selection of the next idle channel;
- preference for reacquisition and assisted acquisition when available.

This design makes receiver behavior predictable under heavy channel configurations. It may take longer to cold-acquire all configured satellites, but once channels are locked the receiver protects tracking continuity.

7.12 Why Tracking Uses Time-Domain Correlators

After acquisition, the channel no longer needs all code phases. It only needs prompt, early, late, noise, and optional monitor taps. A time-domain dot product over a few taps is cheaper than an FFT over all

code phases for each tracking epoch. This is why normal tracking uses `sdr_corr_std()` instead of `sdr_corr_fft()`.

L6 also needs a wide correlation region each symbol because CSK data is encoded as a code-phase shift, but that search runs on chip-binned data with a small fixed-size FFT; the loop correlators still come from the time-domain correlator.

7.13 Avoiding Work in Idle Channels

Idle channels do not run correlators. They only exist as configured objects until the scheduler starts a search. This is important because a typical configuration may include many PRN/signal combinations that are not visible. The permanent cost of an idle channel is memory for code replicas and thread state; the expensive acquisition sum is allocated only during search.

The thread still wakes periodically, but it only advances when the scheduler has changed the channel state or when it is locked. This reduces CPU load compared with a design where every channel continuously tests for signal presence.

7.14 Output Cost Control

The receiver writes logs and streams from the receiver and PVT paths. Output can be expensive if it blocks or if many messages are emitted. The implementation uses stream helpers and emits high-rate channel status only at configured intervals. Raw IF stream output writes the original raw input block rather than re-encoding internal buffers.

For performance-sensitive live use, output configuration matters. RTCM/NMEA/log streams add less CPU than acquisition, but slow file or network streams can still affect latency if they block.

8. Other Implementation Notes

8.1 Input Sources and Tag Files

The receiver can be opened from a Pocket SDR FE device, a SoapySDR device, a local IF file, or a stream. File inputs can use `.tag` sidecar files to recover IF format, sampling frequency, LO frequencies, sampling type, and bit width. When raw IF output is enabled for device input, the receiver writes matching tag files so captured data can be replayed later.

8.2 Real-Sampling Channels

For real-sampled RF channels, the software shifts by `fs/4` during raw sample conversion and subtracts `fs/4` from the channel IF frequency. This keeps later carrier mixing and correlation code on the same complex-sample path used by IQ channels.

8.3 Array Channels

For multi-channel raw FE formats, optional array channels can be produced by combining RF-channel IF buffers with beam-forming weights. These derived channels are treated like additional RF channels by the tracking receiver. The array calibration and beam-control APIs are present in `sdr_rcv.c`, but the core acquisition/tracking algorithms remain the same after the derived IF buffer is generated.

8.4 Status and Diagnostics

Receiver status is derived from the same internal state used by the algorithms: locked channel count, active search channel, buffer usage, C/N0, code offset, Doppler, accumulated Doppler range, synchronization flags, navigation counters, and PVT status. Correlator status and history APIs expose prompt/early/late and extra correlator outputs for visualization and debugging.

8.5 Main Source Files

File	Role in this receiver path
`src/sdr_rcv.c`	Top-level receiver, input, buffering, search scheduling, threads, status, streams
`src/sdr_ch.c`	Channel acquisition, tracking loops, correlators, C/N0, secondary-code sync
`src/sdr_func.c`	FFT/code search, carrier mixing, standard and FFT correlators, SIMD helpers
`src/sdr_code.c`	Primary/secondary code generation, code periods, signal frequencies, code FFTs
`src/sdr_nav.c`	Signal-specific navigation message synchronization and decoding

File	Role in this receiver path
`src/sdr_pvt.c`	Navigation-data ingestion, observation generation, RTCM/NMEA/log output, PVT
`src/sdr_fec.c`, `src/sdr_ldpc.c`, `src/sdr_nb_ldpc.c`	FEC decoders used by navigation decoding
`src/sdr_dev.c`, `src/sdr_sdev.c`, `src/sdr_usb.c`	Device input support

Appendix A. Receiver Constants and Their Algorithmic Roles

The following constants appear as implementation details in the source, but they also define the receiver algorithm's practical behavior.

Constant	Default	Role
`SDR_CYC`	`1 ms`	Main receiver input cycle
`MAX_BUFF`	`8000` cycles	Circular IF buffer length
`TH_CYC`	`50 ms`	Channel thread sleep interval
`LOG_CYC`	`1000` cycles	Receiver time/status log cadence
`SCALE_CYC`	`1000` cycles	Digital scaling update cadence
`TO_REACQ`	`60 s`	Doppler reuse window after loss
`MIN_LOCK`	`2 s`	Minimum lock time before re-acquisition aid
`MAX_ACQ`	`4 ms`	Maximum code period for direct acquisition
`MAX_BUFF_USE`	`90 %`	Search suppression threshold
`SDR_N_CODES`	`10`	Fractional code-phase bank count
`SDR_N_HIST`	`5000`	Prompt-history length
`SP_CORR`	`0.25 chip`	Early/late correlator spacing
`T_ACQ`	`20 ms`	Acquisition non-coherent integration time
`T_ACQ_EXT`	`100 ms`	Assisted-acquisition integration time
`T_COH`	`20 ms`	Requested coherent pilot PLL interval
`T_DLL`	`20 ms`	DLL non-coherent accumulation time
`T_CN0`	`0.5 s`	C/N0 averaging interval
`T_FPULLIN`	`1.0 s`	FLL pull-in duration before PLL
`T_NPULLIN`	`1.5 s`	Navigation decoder pull-in delay
`B_DLL`	`0.25 Hz`	DLL noise bandwidth
`B_PLL`	`5.0 Hz`	PLL noise bandwidth
`B_FLL_W`	`5.0 Hz`	Wide FLL bandwidth
`B_FLL_N`	`2.0 Hz`	Narrow FLL bandwidth
`MAX_DOP`	`5000 Hz`	Default acquisition Doppler search limit
`THRES_CN0_L`	`34 dB-Hz`	Acquisition lock threshold
`THRES_CN0_EXT`	`30 dB-Hz`	Assisted-acquisition threshold
`THRES_CN0_U`	`25 dB-Hz`	Tracking loss threshold
`THRES_CN0_L6`	`32.5 dB-Hz`	L6 tracking loss threshold

These values are tuned for a host-based software receiver. They balance sensitivity, CPU load, latency, and robustness. Increasing acquisition integration time may improve weak-signal detection, but it also extends the time one channel occupies the single active search slot. Increasing loop bandwidths may improve dynamic response, but it increases noise in Doppler, phase, and code observables. Increasing `MAX_BUFF` improves tolerance to CPU bursts at the cost of memory.

Appendix B. Channel Lifecycle

The following table shows the main lifecycle events for one channel.

Event	State before	State after	Main functions
Channel created	none	`IDLE`	`sdr_ch_new()`, `acq_new()`, `trk_new()`
Scheduler starts search	`IDLE`	`SRCH`	`update_srch_ch()`
Search accumulation continues	`SRCH`	`SRCH`	`search_sig()`, `sdr_search_code()`
Signal found	`SRCH`	`LOCK`	`sdr_corr_max()`, `sdr_fine_dop()`, `start_track()`
Signal not found	`SRCH`	`IDLE`	`search_sig()`
Tracking update	`LOCK`	`LOCK`	`track_sig()`
Navigation frame decoded	`LOCK`	`LOCK`	`sdr_nav_decode()`, `sdr_pvt_udnav()`
Observation epoch reached	`LOCK`	`LOCK`	`sdr_pvt_udobs()`
C/N0 loss	`LOCK`	`IDLE`	`track_sig()`
Re-acquisition starts	`IDLE`	`SRCH`	`re_acq()`, `update_srch_ch()`

The lifecycle is intentionally cyclic. A channel is not destroyed when a signal is lost. It returns to idle with enough retained information to support re-acquisition. Long-lived channel objects also make status display stable: channel numbers, signal IDs, PRNs, RF channel assignments, and lost-lock counts remain associated with the same object across search and lock cycles.

Appendix C. Major Call Graphs

C.1 Live Receiver Startup

```
sdr_rcv_open_dev() / sdr_rcv_open_sdev() / sdr_rcv_open_file()
    -> collect IF metadata
    -> sdr_rcv_new()
        -> set_rfch()
        -> ch_th_new()
            -> sdr_ch_new()
                -> sdr_gen_code()
                -> sdr_sec_code()
                -> acq_new()
                -> trk_new()
                -> sdr_nav_new()
        -> sdr_buff_new()
        -> sdr_pvt_new()
    -> sdr_rcv_start()
        -> create channel threads
        -> create receiver thread
```

C.2 Receiver Thread

```
rcv_thread()
    -> read_data()
    -> write_buff()
        -> unpack/scale raw data
        -> apply optional LPF
        -> combine optional array channels
        -> publish buffer index
    -> sdr_str_write(raw stream)
    -> update_srch_ch()
    -> sdr_pvt_udsol()
    -> update_data_stats()
    -> update_scale()
```

C.3 Channel Thread

```
ch_thread()
    -> sdr_ch_update()
        -> search_sig() if SRCH
        -> track_sig()  if LOCK
            -> sdr_corr_std() (carrier mixing fused), or
            -> sdr_mix_carr() -> sdr_corr_std_cpx_code() (E5ABQ), or
            -> sdr_mix_carr() -> sdr_bin_csk() -> chip-domain FFT
               -> sdr_corr_std() with CSK-shifted code (L6D/E)
            -> sync_sec_code()
            -> FLL() or PLL()
            -> DLL()
            -> CN0()
            -> sdr_nav_decode()
    -> sdr_pvt_udnav() if nav->stat
    -> sdr_pvt_udobs()
```

C.4 PVT Epoch Finalization

```
sdr_pvt_udsol()
    -> resolve selected ms ambiguities
    -> sortobs()
    -> out_log_obs()
    -> out_rtcm3_obs()
    -> update_sol()
        -> pntpos()
        -> corr_sol_time()
        -> out_log_pos()
        -> out_nmea()
        -> out_log_sat()
    -> sdr_rcv_array_calib()
    -> advance next epoch
```

These call graphs omit error returns and stream setup, but they show the main algorithmic ownership. When modifying the receiver, this ownership is more important than file boundaries alone. For example, acquisition code lives in `sdr_ch.c` and `sdr_func.c`, but acquisition scheduling lives in `sdr_rcv.c`.

Appendix D. Signal-Specific Behavior Summary

The receiver has a generic acquisition/tracking skeleton, but several signals activate specialized behavior.

Signal family	Special behavior
GLONASS FDMA legacy	PRN argument represents FCN; carrier frequency is shifted
BOC-like signals	Optional bump-jump monitor taps
Pilot signals	Timing may be secondary-code based and ambiguity-resolved
`L6D`, `L6E`	Chip-domain FFT CSK symbol detection, CSK-shifted E/P/L
`E5ABQ`	E5aQ acquisition, AltBOC complex-code tracking banks
GPS/QZSS CNAV-2	LDPC decoding and known SF1 symbol-table sync
Galileo I/NAV/F/NAV	Separate ephemeris ingestion paths
BeiDou D1/D2	Ephemeris consistency check before acceptance
SBAS	SBAS message update path for applicable message types

This table is not a replacement for the source code. It is a map of where the generic receiver model is extended. New signal support should start by deciding which rows it resembles: ordinary data signal, ordinary pilot signal, long-code signal, CSK signal, complex-code signal, or signal requiring a special navigation decoder.

Appendix E. Notes for Extending the Receiver

E.1 Adding a New Ordinary Signal

An ordinary signal with one primary code, optional secondary code, and a data-bit navigation message usually needs changes in these places:

1. code generation in `sdr_code.c`;
2. code period, code length, and nominal carrier frequency helpers;
3. satellite ID mapping if the PRN/system is new;
4. navigation decoder dispatch in `sdr_nav_decode()`;
5. RINEX observation code mapping in the PVT module;
6. signal ID documentation and command reference.

If the code period is no longer than `sdr_max_acq`, the existing acquisition path can usually be reused. If the signal has a BOC-like correlation function, bump-jump spacing may need to be defined. If the signal is pilot-only, the decoder must set TOW validity carefully so that observation generation does not use unresolved time.

E.2 Adding a Long-Code or Special Modulation Signal

Long-code signals can exceed the assumptions of direct acquisition. A new signal may require one of these strategies:

- use a shorter pilot/acquisition-assist component;
- use assisted acquisition from another signal of the same satellite;
- implement a signal-specific folded or partial acquisition path;
- restrict direct acquisition through `sdr_max_acq` and require prior aiding.

Special modulation may also require a complex local replica or a tracking correlator that searches more than prompt/early/late positions. `E5ABQ` and L6 are examples of two different solutions: complex-code banks for AltBOC, and a chip-domain FFT symbol search for CSK.

E.3 Changing Loop Bandwidths

Loop bandwidth changes affect measurement noise, dynamic tolerance, and loss behavior. A wider PLL can follow faster dynamics and oscillator noise but increases carrier-phase noise. A narrower DLL reduces code noise but can lag under dynamics or poor Doppler aiding. Any bandwidth change should be tested with:

- static clean-sky data;
- weak-signal data;
- receiver motion or oscillator drift;

- reacquisition after loss;
- multi-frequency observation consistency.

E.4 Changing the 1 ms Receiver Cycle

The 1 ms cycle appears in buffer indexing, thread scheduling, PVT epoch alignment, prompt-history interpretation, and direct-acquisition assumptions. Changing it would be a receiver architecture change, not a local constant edit. All of these must be reviewed:

- channel thread step calculation `ch->N / rcv->N`;
- circular-buffer capacity and indexing;
- acquisition sample window length;
- file replay timing;
- PVT epoch rounding;
- status/log cadence;
- assumptions in proposed short-code signal support.

E.5 Concurrency Safety

New code should respect the existing ownership model:

- the receiver thread writes IF buffers and publishes `rcv->ix`;
- channel threads read IF buffers and update their own channels;
- PVT updates happen only under `pvt->mtx`;
- channel status reads should lock channel state when reading values that can change during tracking;
- long operations should not hold receiver or PVT mutexes unless they protect data that must remain consistent.

The easiest way to avoid concurrency regressions is to keep expensive signal processing outside shared locks and only lock while copying compact state into shared objects.

Appendix F. Practical Interpretation of Status Fields

The console and API status fields are direct views into receiver state.

Status item	Interpretation
`LOCK(s)`	`ch->lock * ch->T`, elapsed tracked time since current lock
`C/N0`	Filtered C/N0 estimate from prompt/noise correlator powers
`COFF(ms)`	Code offset inside the current primary-code period
`DOP(Hz)`	Current Doppler estimate
`ADR(cyc)`	Accumulated Doppler range used for carrier phase
`SYNC S`	Tracking secondary-code sync
`SYNC B`	Navigation symbol sync
`SYNC F`	Navigation frame sync
`SYNC R`	Reversed navigation polarity
`#NAV`	Successful navigation decode count
`#ERR`	Failed navigation decode count
`#LOL`	Loss-of-lock count
`FEC`	Number of corrected FEC errors for the last decoded frame

These fields are useful for diagnosing where a receiver problem lies. A channel with high C/N0 but no frame sync is usually a navigation-decoder or signal selection issue. A channel with repeated acquisition and loss may have an IF frequency, RF channel, loop bandwidth, or C/N0 problem. A channel with stable tracking but no PVT contribution may have unresolved TOW, missing ephemeris, or ambiguous pseudorange.

Appendix G. Mathematical Details

G.1 Sampled IF Model

After front-end sampling and software unpacking, one RF channel buffer contains samples that can be modeled as:

$$x[k] = A\, b[k]\, c[k - k_\tau]\, \exp(j(2\pi f_{\mathrm{IF}} \frac{k}{f_s} + \phi)) + w[k]$$

where `b[k]` represents data or secondary-code polarity over the current interval, `c[k]` is the local spreading-code sequence sampled at `fs`, and `k_tau` is the code delay in samples. The receiver does not estimate `A` directly. It estimates code delay, carrier frequency, carrier phase, and C/N0 from correlations.

Carrier wipeoff multiplies by the conjugate carrier replica:

$$y[k] = x[k]\, \exp(-j(2\pi f_{\mathrm{rep}} \frac{k}{f_s} + \phi_{\mathrm{rep}}))$$

If `f_rep` is close to the true IF plus Doppler, the desired signal becomes nearly stationary over one code period and can be integrated by code correlation. The residual phase rotation is what the FLL/PLL estimates.

G.2 Parallel Code Search

For a fixed Doppler bin, acquisition computes the circular correlation between the carrier-wiped data and the local code:

$$R[m] = \sum_k y[k]\, \mathrm{conj}(c[k - m])$$

Direct evaluation for all `m` would be expensive. The PCPS method uses the FFT:

$$R = \mathrm{IFFT}(\mathrm{FFT}(y)\, \mathrm{conj}(\mathrm{FFT}(c)))$$

The implementation stores `conj(FFT(c))` in `code_fft`. During acquisition it only has to FFT the Doppler-wiped data, multiply by `code_fft`, and inverse FFT the result. Zero padding to `2 * N` reduces ambiguity around the block boundary and supports a full code-offset search using the two-code-period input window.

The receiver accumulates:

$$P_{\mathrm{sum}}[f_d, m] \leftarrow P_{\mathrm{sum}}[f_d, m] + |R_{f_d}[m]|^2$$

This non-coherent sum is robust against unknown bit or secondary-code polarity between code periods. The acquisition peak is selected from `P_sum`.

G.3 C/N0 from Correlation Powers

The acquisition metric and tracking C/N0 estimate both compare signal-related power with a noise-like reference, but they use different references.

In acquisition, the average over the Doppler/code search grid approximates the background correlation power. The peak excess is interpreted as signal power:

$$(C/N_0)_{\text{acq}} = 10\log_{10}\left(\frac{P_{\max} - P_{\text{ave}}}{P_{\text{ave}}\,T}\right)$$

In tracking, the receiver has a prompt correlator and a deliberately displaced noise correlator. Their powers are accumulated over 0.5 s. The noise reference is subtracted from the prompt before forming the ratio:

$$P_S = \max\left(\sum |P|^2 - \sum |N|^2,\ 0.01\sum |N|^2\right)$$

$$(C/N_0)_{\text{trk}} = 10\log_{10}\left(\frac{P_S}{T\sum |N|^2}\right)$$

The subtraction removes the `10*log10(1/T)` low-C/N0 floor of the former prompt/noise ratio. Both are practical receiver metrics. They are not identical estimators and should not be expected to match exactly at lock transition. The acquisition metric is used for detection; the tracking metric is used for status and loss detection.

G.4 Early-Late DLL Shape

The early-minus-late discriminator assumes that the prompt lock point lies near the peak of the code autocorrelation. With early and late taps symmetric around prompt, the normalized error:

$$\frac{E - L}{E + L}$$

is near zero at the lock point. Multiplying by `0.5 * T / code_length` converts the normalized value to seconds using the local chip interval. The discriminator is robust because it uses magnitudes, not prompt sign. The implementation still uses prompt sign in `sumI[]` to provide data-wiped in-phase monitor values for status and visualization.

BOC signals can have multiple correlation peaks. The standard early-late DLL can lock on a side peak if acquisition or transient dynamics place the code offset there. The optional bump-jump logic adds very-early and very-late taps to detect such false locks and shift the code offset toward the main peak.

G.5 Carrier Loop Sign Conventions

The receiver's carrier loops update Doppler `fd`; accumulated carrier phase for observations is derived from `adr`. A positive `fd` increases `adr`, and the carrier-phase observable is generated as `-adr` plus phase-alignment terms.

This convention is internally consistent with the carrier wipeoff and RTKLIB observation output. When comparing Pocket SDR Doppler or carrier phase with another receiver, sign conventions must be checked at the observation format level rather than inferred only from loop equations.

G.6 Time Ambiguity

Pseudorange requires absolute transmit time relative to receiver time. A channel can know code phase very precisely but still not know which millisecond, 20-millisecond, 100-millisecond, or longer epoch it belongs to. Navigation decoding resolves this ambiguity by extracting TOW. Pilot-only channels may need a companion signal.

The implementation represents this with `tow_v`:

```
0: no valid TOW
1: valid absolute TOW
2: ambiguous TOW that may be resolved
```

This is a compact but important design. It prevents the tracking layer from claiming absolute timing prematurely, and it gives the PVT layer a chance to resolve ambiguity using multi-signal observations.

Appendix H. Detailed Implementation Review Checklist

H.1 Receiver Input and Buffering

Check these points when editing input or buffer code:

- Does the code still publish `rcv->ix` only after a full 1 ms block is written?
- Does circular-buffer wraparound preserve carrier and `fs/4` phase continuity?
- Are packed FE channels unpacked into the correct RF buffer?
- Are I-only and IQ channels both represented as complex internal samples?
- Does any new low-pass or scaling operation run in place only on the intended RF channel?
- Does raw IF logging preserve the original byte stream and matching tag metadata?
- Can file replay still sleep according to `tscale` without blocking live device paths?

Buffer bugs often show up as apparently random acquisition failures across many signals. If all signals on one RF channel fail, inspect RF-channel assignment, LO metadata, unpacking, and scaling before inspecting navigation decoders.

H.2 Acquisition

Check these points when editing acquisition:

- Is the coherent integration interval still one primary code period?
- Is the input data length sufficient for the chosen FFT correlation length?
- Is `P_sum` cleared or freed after every acquisition decision?
- Does assisted acquisition still reduce Doppler bins without losing the correct center frequency?
- Does failed acquisition return the channel to idle?
- Does successful acquisition initialize `fd`, `coff`, `cn0`, `lock`, `week`, `tow`, tracking state, and navigation state?
- Does the scheduler still prevent multiple simultaneous searches?

Acquisition changes should be tested with both cold acquisition and re-acquisition. A change that works for cold acquisition can still break re-acquisition if it ignores `fd_ext` or stale `P_sum` state.

H.3 Tracking

Check these points when editing tracking:

- Is carrier phase continuous across epochs?
- Is `adr` updated before observation generation can read it?
- Is code offset wrapped consistently with lock count and TOW?
- Does secondary-code wipeoff occur before navigation symbol generation?
- Are prompt correlations appended exactly once per code epoch?

- Are FLL/PLL pull-in transitions still based on `lock * T`?
- Is the loss test using filtered C/N0 rather than one noisy prompt sample?
- Are signal-specific paths such as L6 and E5ABQ still producing normal `trk->C[]` outputs for the common loop code?

Tracking bugs often appear as unstable Doppler, periodic loss, no frame sync, or large carrier-phase discontinuities. The prompt history and lock-count handling should be inspected early in such cases.

H.4 Navigation Decoding

Check these points when editing navigation decoders:

- Does the decoder wait for enough symbols before accessing the history buffer?
- Does it support reversed polarity if the signal can appear inverted?
- Does it preserve soft-symbol magnitude for FEC paths and use local hard decisions only where bit-level parity, BCH, or CRC logic requires them?
- Does it validate parity, CRC, or FEC before setting `nav->stat`?
- Does it update week and TOW only from valid message fields?
- Does it set `tow_v` correctly for data and pilot cases?
- Does it clear sync and TOW on frame mismatch?
- Does it increment success and failure counters consistently?
- Does the packed `nav->data` layout match the PVT ingestion decoder?

Navigation decoder bugs can be subtle because tracking can look perfect. A channel with stable C/N0 and prompt history but no PVT contribution should be checked for symbol sync, frame sync, CRC/FEC success, and TOW validity.

H.5 Observation and PVT

Check these points when editing observation or PVT code:

- Are observations generated only at the current PVT epoch?
- Does `update_obs()` reject invalid satellite IDs and invalid pseudoranges?
- Are pilot ambiguities invalidated when no companion observation exists?
- Are carrier-phase alignment constants applied only to the intended signals?
- Are RTKLIB code indices and receiver numbers assigned consistently?
- Are navigation updates protected by the PVT mutex?
- Does solution failure avoid disturbing tracking state?
- Are output streams optional and checked for valid stream handles?

PVT changes should be tested with single-system and multi-system data. They should also be tested with missing ephemerides and partial tracking, because the receiver must continue running even when no position solution is available.

Appendix I. Troubleshooting by Symptom

I.1 No Signals Acquired

Likely causes:

- wrong IF format or sample rate;
- wrong LO frequency or RF channel mapping;
- wrong I/Q convention;
- front-end not streaming or file ended;
- Doppler range too narrow;
- input scaling too small or clipped;
- antenna or front-end failure.

Useful checks:

- receiver data rate and buffer usage;
- RF-channel PSD and histogram APIs;
- `.tag` metadata for file input;
- acquisition logs showing `SIGNAL NOT FOUND`;
- known strong L1 C/A signal with a wide Doppler search.

I.2 Some Bands Acquire, Others Do Not

Likely causes:

- per-RF-channel LO metadata is wrong;
- `-RFCH` assignment points a signal to the wrong RF channel;
- real/IQ sampling type is wrong for one channel;
- per-channel bit width is wrong for packed raw input;
- front-end configuration did not enable the expected band.

This symptom usually points to the RF-front-end split rather than the generic acquisition algorithm. The same `sdr_ch.c` acquisition path is used after the IF buffer has been generated.

I.3 Signal Acquires but Quickly Loses Lock

Likely causes:

- acquisition Doppler is near the edge of a bin and FLL cannot pull in;
- carrier loop bandwidth is too narrow for dynamics or oscillator error;
- code offset is a side peak for a BOC signal;
- C/N0 is near the loss threshold;

- IF scaling clips or underuses the internal range;
- wrong signal ID uses the wrong code or code period.

Useful checks:

- Doppler evolution after `SIGNAL FOUND`;
- C/N0 trend during the first second;
- bump-jump logs for BOC signals;
- prompt/early/late correlator history;
- whether assisted acquisition improves lock stability.

I.4 Stable Tracking but No Navigation Decode

Likely causes:

- data/pilot signal confusion;
- frame polarity not handled;
- prompt sign inverted by wrong I/Q convention;
- C/N0 is adequate for tracking but not for decoding;
- decoder expects a different PRN/system mode;
- secondary-code sync is not established for a pilot-timed path.

Useful checks:

- `SYNC` flags: `S`, `B`, `F`, and `R`;
- `#NAV` and `#ERR` counters;
- `$NAV` logs;
- prompt I sign history;
- signal-specific decoder path in `sdr_nav_decode()`.

I.5 Navigation Decodes but No PVT

Likely causes:

- too few satellites with valid pseudorange;
- ephemeris not decoded for tracked satellites;
- PVT epoch not initialized;
- pilot ambiguities unresolved;
- elevation mask too high;
- RAIM-FDE rejects the observation set;
- multi-system clock handling lacks enough satellites.

Useful checks:

- `$OBS` log count and satellite list;
- `$EPH` or navigation update logs;

- PVT error messages from `pntpos()`;
- `pvt->count[]` solution/observation/navigation counters;
- whether L1-like pseudoranges exist for enough satellites.

I.6 PVT Jumps or Has Large Bias

Likely causes:

- wrong sample rate or LO frequency causing biased code tracking;
- unresolved millisecond ambiguity used as a range;
- wrong signal-to-RINEX code mapping;
- ephemeris mismatch or stale ephemeris;
- receiver clock epoch adjustment issue;
- incorrect phase/code alignment for a newly added signal.

Useful checks:

- per-satellite pseudorange residuals in RTKLIB traces;
- companion-signal ambiguity resolution;
- navigation message time fields;
- solution clock offset and epoch adjustment logs;
- comparison against a known-good RINEX conversion.

Appendix J. Design Rationale

J.1 Why One Channel per Signal/PRN/RF Channel

The receiver could have been designed around satellites as top-level objects, with multiple signal components under each satellite. Pocket SDR instead uses one independent channel per `(signal, PRN, RF channel)` combination. This keeps acquisition and tracking uniform: every channel has one primary code, one IF frequency, one Doppler estimate, one code offset, and one navigation decoder state.

The tradeoff is that cross-signal relationships are handled indirectly. Doppler assistance searches for another locked channel with the same satellite ID. PVT merges observations by satellite ID and observation code. Pilot ambiguity is resolved from companion observations. This design is simple and flexible, but new multi-component signals may require careful coordination through these existing mechanisms.

J.2 Soft Navigation Symbols

The navigation decoders use a single rolling `nav->syms` buffer for binary soft symbols. This avoids maintaining parallel hard and soft buffers while letting FEC paths use reliability information from the prompt correlation. Hard-decision paths convert the soft symbols locally when a signal definition requires bit-level parity, BCH, CRC, or table matching.

This design keeps the navigation buffer compact and lets preamble and known-symbol-table matching use soft scores without a parallel buffer. L6 CSK remains a signal-specific exception because its symbols are code-shift indices rather than binary prompt-correlation signs.

J.3 Why PVT Uses Single-Point Positioning

The real-time receiver path produces raw observations and navigation data, but its built-in PVT solution is single-point positioning. This is appropriate for receiver status, live display, and sanity checks. Higher-precision processing can use the logged observations, RTCM stream, or converted RINEX data outside the receiver.

This separation keeps the receiver focused on signal processing and basic navigation. It avoids coupling channel tracking stability to a complex estimation filter. If point positioning fails, tracking continues and raw outputs remain available.

J.4 Why Acquisition Is Conservative

The receiver does not attempt every possible acquisition optimization. It uses a robust PCPS method with non-coherent accumulation and a single active search slot. This is conservative, but it matches the real-time receiver requirement: locked channels should not be sacrificed to speed up cold search.

More aggressive acquisition methods can be added for specific signals, but they should preserve three properties:

- bounded CPU use under large channel configurations;
- clear handoff to the common tracking state;
- safe behavior when the aiding information is missing or wrong.

Appendix K. Minimal Test Matrix for Algorithm Changes

A useful regression test matrix for receiver algorithm changes should include:

Area	Test case
Input conversion	Replay known IF files for `INT8X2`, `RAW16`, and `RAW32` if available
Acquisition	Cold-acquire GPS L1 C/A, Galileo E1B, and one long/modernized signal
Re-acquisition	Force or wait for loss, confirm Doppler-assisted search starts
Tracking	Compare Doppler, C/N0, and lock time over a fixed data segment
Navigation	Confirm `$NAV` output and ephemeris update for each affected system
Observation	Confirm `$OBS` contains expected RINEX codes and valid pseudoranges
PVT	Confirm single-point solution and solution latency remain reasonable
Pilot ambiguity	Confirm ambiguous pilot observations are resolved or invalidated
L6/CSK	Confirm CSK symbol detection if L6 code paths are touched
E5ABQ	Confirm bank selection after secondary-code sync if AltBOC code is touched

For code changes with limited scope, not every row is mandatory, but the rows that touch the changed data path should be exercised. For example, a change in sample conversion should run acquisition and tracking tests, while a change in navigation decoding should focus on `$NAV`, ephemeris ingestion, observation time validity, and PVT contribution.

References

- `src/sdr_rcv.c`, `src/sdr_ch.c`, `src/sdr_func.c`, `src/sdr_nav.c`, `src/sdr_pvt.c`, and `src/sdr_code.c`.
- `doc/command_ref.md`, especially the `pocket_trk` and signal-ID sections.
- `doc/IPNTJ_Seminar_202605revB.pdf` for implementation overview figures and seminar material related to the current Pocket SDR receiver.